```
// packages/shared/src/config.ts
/**
 * 对局配置（config-as-data）。默认值 = 方案 5.9 GDD 数值框架 v1 + v0.7 修正。
 * 所有对局数值只允许出现在这里（及服务端下发的覆盖配置）；core 不得硬编码数值。
 */
import type { Tier } from './live-events';
export type SideId = 'A' | 'B';
export const SIDES: readonly SideId[] = ['A', 'B'];
export function otherSide(side: SideId): SideId {
  return side === 'A' ? 'B' : 'A';
}
/** 档位效果。克制环：加速 → 克爆发；护盾 → 克加速；爆发（精英锦鲤/神龙）→ 克护盾。 */
export type TierEffect =
  | { kind: 'fry' }
  | { kind: 'accel'; boost: number; durationMs: number }
  | { kind: 'shield'; durationMs: number }
  | { kind: 'elite'; suppressFry: number; durationMs: number }
  | { kind: 'strengthen'; boost: number; durationMs: number }
  | { kind: 'dragon'; suppress: number; durationMs: number }
  | { kind: 'leap' };
export interface TierRule {
  /** 助力值（贡献分）：用于榜单、归因、成就。与价值成正比。 */
  power: number;
  /** 进度（胜负推进），乘数前基数。T5+ 受上限约束（方案 5.9 ⑧）。 */
  progress: number;
  /** instant = 立即推进；flow = 在 flowMs 内匀速兑现（鱼苗单位）。 */
  delivery: 'instant' | 'flow';
  flowMs: number;
  effect: TierEffect;
  /** 外显描述（≤5 字，过词表）。 */
  label: string;
}
export interface SkinRule {
  id: string;
  label: string;
  /** 概率权重（公示值，合计 100）。 */
  weight: number;
  /** 稀有款触发全屏演出。 */
  rare: boolean;
}
export interface GameConfig {
  version: string;
  sides: Record<SideId, { name: string; joinKeywords: string[] }>;
  phases: {
    /** 蓄势时长（人数充足）。 */
    chargeMs: number;
    /** 蓄势时长（空场弹性：参与人数低于阈值时）。 */
    chargeMsSparse: number;
    sparseThreshold: number;
    clashMs: number;
```

```
  fortuneMs: number;
  settleMs: number;
};
/** 胜负线：进度当量/边。 */
winProgress: number;
/** 各阶段进度上限（占胜负线比例）：节奏前松后紧（方案 5.9 ①：蓄势 ≤5%、攻防 ≤85%、鸿运起可达 100%）。 */
progressCeil: { charge: number; clash: number };
/** 鸿运时刻全效果倍率。 */
fortuneMultiplier: number;
/** 终局潮汐：鸿运结束仍未分胜负时双方等速自动推进（进度/秒）。 */
overtimeRatePerSec: number;
/** 下标 = 档位 T0~T7。 */
tiers: [TierRule, TierRule, TierRule, TierRule, TierRule, TierRule, TierRule, TierRule];
randomSkin: {
  /** 未被指定为随机外观款的 T4 价值礼物按此档位结算效果。 */
  fallbackTier: Tier;
  skins: SkinRule[];
};
dragon: { perMatch: number; perUser: number; fallbackTier: Tier };
/** 大额软性防鲸：单用户本局第 fromNth 件 ≥fromTier 起，个人推进部分按 personalShare，差额转全队匀速兑现。 */
whale: { fromTier: Tier; fromNth: number; personalShare: number; teamFlowMs: number };
free: {
  likeProgress: number;
  likeCharge: number;
  likePerUserPerSecCap: number;
  commentProgress: number;
  commentCharge: number;
  commentCooldownMs: number;
};
charge: {
  cap: number;
  burstProgress: number;
  giftChargePerPower: number;
  /** 每个充能周期内礼物贡献的充能上限占比（可加速不可替代）。 */
  giftShareCap: number;
};
balance: {
  exponent: number;
  maxWeight: number;
  comebackOn: number;
  comebackOff: number;
  comebackBoost: number;
  lossStreakStep: number;
  lossStreakCap: number;
};
/** 克制命中时被克方效果乘数。 */
counterFactor: number;
settle: { topN: number; starsBase: number; starsWinBonus: number };
/** 配置热更新生效策略。 */
hotReload: 'next-match' | 'immediate';
```

```
}
const SECOND = 1000;
export const DEFAULT_CONFIG: GameConfig = {
 version: '5.9-v1+v0.7',
 sides: {
  A: { name: '锦鲤', joinKeywords: ['锦鲤', '1'] },
  B: { name: '神龟', joinKeywords: ['神龟', '2'] },
 },
 phases: {
  chargeMs: 120 * SECOND,
  chargeMsSparse: 60 * SECOND,
  sparseThreshold: 10,
  clashMs: 480 * SECOND,
  fortuneMs: 180 * SECOND,
  settleMs: 100 * SECOND,
 },
 winProgress: 8000,
 progressCeil: { charge: 0.05, clash: 0.85 },
 fortuneMultiplier: 2,
 overtimeRatePerSec: 40,
 tiers: [
  { power: 25, progress: 25, delivery: 'flow', flowMs: 60 * SECOND, effect: { kind: 'fry' }, label: '召唤鱼群' },
  { power: 100, progress: 100, delivery: 'flow', flowMs: 60 * SECOND, effect: { kind: 'fry' }, label: '鱼群+1' },
  { power: 260, progress: 260, delivery: 'instant', flowMs: 0, effect: { kind: 'accel', boost: 0.15, durationMs: 10 * SECOND }, label: '全队加速' },
  { power: 520, progress: 520, delivery: 'instant', flowMs: 0, effect: { kind: 'shield', durationMs: 20 * SECOND }, label: '挡一次' },
  { power: 1050, progress: 1000, delivery: 'instant', flowMs: 0, effect: { kind: 'elite', suppressFry: 0.3, durationMs: 45 * SECOND }, label: '召唤锦鲤' },
  { power: 2600, progress: 1000, delivery: 'instant', flowMs: 0, effect: { kind: 'strengthen', boost: 0.2, durationMs: 30 * SECOND }, label: '全队强化' },
  { power: 5200, progress: 800, delivery: 'instant', flowMs: 0, effect: { kind: 'dragon', suppress: 0.25, durationMs: 8 * SECOND }, label: '召唤神龙' },
  { power: 13000, progress: 960, delivery: 'instant', flowMs: 0, effect: { kind: 'leap' }, label: '鲤跃龙门' },
 ],
 randomSkin: {
  fallbackTier: 3,
  skins: [
   { id: 'gold', label: '金鲤', weight: 70, rare: false },
   { id: 'rainbow', label: '彩虹鲤', weight: 25, rare: false },
   { id: 'star', label: '星耀鲤', weight: 5, rare: true },
  ],
 },
 dragon: { perMatch: 3, perUser: 1, fallbackTier: 5 },
 whale: { fromTier: 5, fromNth: 2, personalShare: 0.7, teamFlowMs: 30 * SECOND },
 free: {
  likeProgress: 0.5,
  likeCharge: 1,
  likePerUserPerSecCap: 15,
  commentProgress: 2,
  commentCharge: 5,
  commentCooldownMs: 3 * SECOND,
 },
 charge: { cap: 400, burstProgress: 350, giftChargePerPower: 0.2, giftShareCap: 0.5 },
 balance: {
```

（著作权人姓名，提交前由人填写）

```
    exponent: 0.4,
    maxWeight: 1.35,
    comebackOn: 0.2,
    comebackOff: 0.1,
    comebackBoost: 1.25,
    lossStreakStep: 0.05,
    lossStreakCap: 0.1,
  },
  counterFactor: 0.5,
  settle: { topN: 5, starsBase: 3, starsWinBonus: 2 },
  hotReload: 'next-match',
};
/** 深拷贝默认配置并按需覆盖顶层字段（测试与调参用）。 */
export function makeConfig(overrides: Partial<GameConfig> = {}): GameConfig {
  const base = JSON.parse(JSON.stringify(DEFAULT_CONFIG)) as GameConfig;
  return { ...base, ...overrides };
}
// packages/shared/src/index.ts
export * from './live-events';
export * from './config';
export * from './wordlist-data';
export * from './text-filter';
export * from './prng';
export * from './sim-script';
// packages/shared/src/live-events.ts
/**
 * 统一直播互动事件模型（LiveEvent）与平台适配器接口。
 *
 * 这是 server / core / client 三端唯一的事件类型来源。任何平台（本地抓取流、
 * 抖音官方推送、快手官方推送……）都必须先经适配器归一化为 LiveEvent，
 * core 只认识 LiveEvent，不认识任何平台。
 */
/** 礼物价值档位 T0~T7。效果只挂在档位上，不绑定任何平台礼物 ID。 */
export type Tier = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7;
export const TIERS: readonly Tier[] = [0, 1, 2, 3, 4, 5, 6, 7];
/** 直播间用户（已由管道完成昵称过滤与截断，可直接展示）。 */
export interface LiveUser {
  /** 平台内用户标识（仅用于房间内归属，不做跨平台归并）。 */
  id: string;
  /** 展示昵称：已过词表过滤，命中禁用词时为打码形态。 */
  nick: string;
  /** 粉丝团等级（平台提供时）。 */
  fansLevel?: number;
}
interface LiveEventBase {
  /** 幂等键：同一来源的同一条消息必须得到相同的键。 */
  msgId: string;
  /** 事件发生时间（毫秒时间戳），core 以此驱动时间。 */
  ts: number;
  roomId: string;
```

```
  user: LiveUser;
}
/** 弹幕。text 已过词表过滤。 */
export interface CommentEvent extends LiveEventBase {
  type: 'comment';
  text: string;
}
/** 点赞（一条消息可能聚合多次点赞）。 */
export interface LikeEvent extends LiveEventBase {
  type: 'like';
  count: number;
}
/**
 * 礼物。只携带档位，不携带金额——金额止于管道（见 live-event-pipeline 规格）。
 */
export interface GiftEvent extends LiveEventBase {
  type: 'gift';
  tier: Tier;
  /** 本条消息代表的礼物个数（连击已按增量拆分）。 */
  count: number;
  /** 是否为本平台唯一指定的随机外观款（模式 A）。 */
  randomSkinGift: boolean;
}
export interface FollowEvent extends LiveEventBase {
  type: 'follow';
}
export interface JoinEvent extends LiveEventBase {
  type: 'join';
}
export type LiveEvent = CommentEvent | LikeEvent | GiftEvent | FollowEvent | JoinEvent;
export type LiveEventType = LiveEvent['type'];
/** 适配器连接状态。 */
export type AdapterStatus =
  | { state: 'connecting' }
  | { state: 'connected' }
  | { state: 'disconnected'; reason: string; retryInMs: number }
  | { state: 'closed' };
/**
 * 平台适配器统一接口。每个平台一个实现；新增平台不得修改 core。
 */
export interface PlatformAdapter {
  /** 适配器名称，如 "local-grab"、"sim"、"douyin"。 */
  readonly name: string;
  /** 建立连接并开始产出事件。 */
  connect(): Promise<void>;
  /** 断开连接并停止产出事件。 */
  close(): Promise<void>;
  /** 订阅归一化后的事件，返回取消订阅函数。 */
  onEvent(handler: (event: LiveEvent) => void): () => void;
  /** 订阅连接状态变化，返回取消订阅函数。 */
```

```
  onStatus(handler: (status: AdapterStatus) => void): () => void;
}
// packages/shared/src/prng.ts
/**
 * 确定性伪随机：同一种子永远得到同一序列（对局可回放）。
 * core 禁用 Math.random()，所有"随机外观"均以事件幂等键为种子。
 */
/** FNV-1a 32 位字符串哈希。 */
export function hashString(s: string): number {
  let h = 0x811c9dc5;
  for (let i = 0; i < s.length; i++) {
    h ^= s.charCodeAt(i);
    h = Math.imul(h, 0x01000193);
  }
  return h >>> 0;
}
/** mulberry32：返回 [0,1) 均匀分布的生成器。 */
export function mulberry32(seed: number): () => number {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
/** 按权重确定性选取。weights 全为 0 时返回 0。 */
export function pickWeighted(weights: readonly number[], r: number): number {
  const total = weights.reduce((a, b) => a + b, 0);
  if (total <= 0) return 0;
  let x = r * total;
  for (let i = 0; i < weights.length; i++) {
    x -= weights[i] ?? 0;
    if (x < 0) return i;
  }
  return weights.length - 1;
}
// packages/shared/src/schema.ts
/**
 * 对局配置运行时校验（zod）——仅 server/tools 使用，不同步进客户端。
 * 底部编译期断言保证 schema 与手写 GameConfig 类型完全一致。
 */
import { z } from 'zod';
import type { GameConfig } from './config';
const tier = z.union([0, 1, 2, 3, 4, 5, 6, 7].map((n) => z.literal(n)) as unknown as [
  z.ZodLiteral<0>, z.ZodLiteral<1>, z.ZodLiteral<2>, z.ZodLiteral<3>,
  z.ZodLiteral<4>, z.ZodLiteral<5>, z.ZodLiteral<6>, z.ZodLiteral<7>,
]);
const ms = z.number().int().nonnegative();
```

```
const ratio = z.number().min(0).max(1);
const pos = z.number().positive();
const effect = z.discriminatedUnion('kind', [
 z.object({ kind: z.literal('fry') }),
 z.object({ kind: z.literal('accel'), boost: ratio, durationMs: ms }),
 z.object({ kind: z.literal('shield'), durationMs: ms }),
 z.object({ kind: z.literal('elite'), suppressFry: ratio, durationMs: ms }),
 z.object({ kind: z.literal('strengthen'), boost: ratio, durationMs: ms }),
 z.object({ kind: z.literal('dragon'), suppress: ratio, durationMs: ms }),
 z.object({ kind: z.literal('leap') }),
]);
const tierRule = z.object({
 power: pos,
 progress: pos,
 delivery: z.enum(['instant', 'flow']),
 flowMs: ms,
 effect,
 label: z.string().min(1).max(5),
});
const side = z.object({ name: z.string().min(1), joinKeywords: z.array(z.string().min(1)).min(1) });
export const gameConfigSchema = z
 .object({
  version: z.string(),
  sides: z.object({ A: side, B: side }),
  phases: z.object({
   chargeMs: ms,
   chargeMsSparse: ms,
   sparseThreshold: z.number().int().nonnegative(),
   clashMs: ms,
   fortuneMs: ms,
   settleMs: ms,
  }),
  winProgress: pos,
  progressCeil: z.object({ charge: ratio, clash: ratio }),
  fortuneMultiplier: z.number().min(1),
  overtimeRatePerSec: pos,
  tiers: z.tuple([tierRule, tierRule, tierRule, tierRule, tierRule, tierRule, tierRule, tierRule]),
  randomSkin: z.object({
   fallbackTier: tier,
   skins: z.array(z.object({ id: z.string(), label: z.string(), weight: z.number().nonnegative(), rare: z.boolean() })).min(1),
  }),
  dragon: z.object({ perMatch: z.number().int().nonnegative(), perUser: z.number().int().nonnegative(), fallbackTier: tier }),
  whale: z.object({ fromTier: tier, fromNth: z.number().int().min(1), personalShare: ratio, teamFlowMs: ms }),
  free: z.object({
   likeProgress: z.number().nonnegative(),
   likeCharge: z.number().nonnegative(),
   likePerUserPerSecCap: z.number().int().positive(),
   commentProgress: z.number().nonnegative(),
   commentCharge: z.number().nonnegative(),
   commentCooldownMs: ms,
```

```
  }),
  charge: z.object({ cap: pos, burstProgress: pos, giftChargePerPower: z.number().nonnegative(), giftShareCap: ratio }),
  balance: z.object({
   exponent: z.number().nonnegative(),
   maxWeight: z.number().min(1),
   comebackOn: ratio,
   comebackOff: ratio,
   comebackBoost: z.number().min(1),
   lossStreakStep: ratio,
   lossStreakCap: ratio,
  }),
  counterFactor: ratio,
  settle: z.object({ topN: z.number().int().positive(), starsBase: z.number().int().nonnegative(), starsWinBonus: z.number().int().nonnegative() }),
  hotReload: z.enum(['next-match', 'immediate']),
 })
 .superRefine((c, ctx) => {
  // 方案 5.9 ⑧：任何单件礼物的进度不得超过胜负线 12.5%
  c.tiers.forEach((t, i) => {
   if (t.progress > c.winProgress * 0.125) {
    ctx.addIssue({ code: z.ZodIssueCode.custom, path: ['tiers', i, 'progress'], message: '单件礼物进度超过胜负线 12.5%（大额不得独立决胜）' });
   }
  });
  if (c.balance.comebackOff >= c.balance.comebackOn) {
   ctx.addIssue({ code: z.ZodIssueCode.custom, path: ['balance'], message: '逆流解除阈值必须小于触发阈值' });
  }
  if (c.tiers[4].effect.kind !== 'elite') {
   ctx.addIssue({ code: z.ZodIssueCode.custom, path: ['tiers', 4], message: 'T4 必须为随机外观款（精英锦鲤）' });
  }
 });
type SchemaType = z.infer<typeof gameConfigSchema>;
type Equals<A, B> = (<T>() => T extends A ? 1 : 2) extends <T>() => T extends B ? 1 : 2 ? true : false;
const schemaMatchesType: Equals<SchemaType, GameConfig> = true;
void schemaMatchesType;
export function parseGameConfig(input: unknown): GameConfig {
 return gameConfigSchema.parse(input) as GameConfig;
}
// packages/shared/src/sim-script.ts
/**
 * 确定性模拟事件剧本：无真实直播间时驱动一局完整演示（demo 模式 / 联调 / 压测）。
 * 同一参数永远生成同一事件序列。
 */
import type { LiveEvent, Tier } from './live-events';
import { mulberry32, pickWeighted } from './prng';
export interface SimOptions {
 seed: number;
 roomId: string;
 startTs: number;
 durationMs: number;
 /** 模拟观众数。 */
 audience: number;
```

```
 /** 每秒平均事件数。 */
 eventsPerSec: number;
 /** 礼物在事件中的占比（0~1）。 */
 giftRatio: number;
}
export const DEFAULT_SIM: Omit<SimOptions, 'startTs'> = {
 seed: 20261016,
 roomId: 'demo-room',
 durationMs: 15 * 60 * 1000,
 audience: 40,
 eventsPerSec: 4,
 giftRatio: 0.04,
};
/** 档位分布：小额高频、大额稀少（与方案 5.6 付费分布方向一致）。 */
const TIER_WEIGHTS: Record<Tier, number> = { 0: 45, 1: 22, 2: 12, 3: 9, 4: 6, 5: 3.5, 6: 1.8, 7: 0.7 };
const NICK_HEADS = ['小', '阿', '老', '大', '甜', '酷', '静', '暖'];
const NICK_TAILS = ['鱼', '橙', '星', '叶', '云', '月', '风', '桃', '舟', '禾'];
const SLOGANS = ['冲！', '加油', '稳住', '好运来', '跃龙门', '锦鲤附体'];
export function simNick(i: number): string {
 const head = NICK_HEADS[i % NICK_HEADS.length] ?? '小';
 const tail = NICK_TAILS[Math.floor(i / NICK_HEADS.length) % NICK_TAILS.length] ?? '鱼';
 return `${head}${tail}${i}`;
}
export function generateSimEvents(opts: SimOptions): LiveEvent[] {
 const rand = mulberry32(opts.seed);
 const events: LiveEvent[] = [];
 const tierWeights = Object.values(TIER_WEIGHTS);
 let seq = 0;
 const id = () => `sim-${opts.seed}-${seq++}`;
 const user = (i: number) => ({ id: `u${i}`, nick: simNick(i) });
 // 开场：观众陆续弹幕选阵营
 for (let i = 0; i < opts.audience; i++) {
  const ts = opts.startTs + Math.floor(rand() * 60_000);
  const keyword = rand() < 0.55 ? '锦鲤' : '神龟';
  events.push({ type: 'comment', msgId: id(), ts, roomId: opts.roomId, user: user(i), text: keyword });
 }
 const total = Math.floor((opts.durationMs / 1000) * opts.eventsPerSec);
 for (let k = 0; k < total; k++) {
  const ts = opts.startTs + Math.floor(rand() * opts.durationMs);
  const u = user(Math.floor(rand() * opts.audience));
  const r = rand();
  if (r < opts.giftRatio) {
   const tier = pickWeighted(tierWeights, rand()) as Tier;
   const count = tier === 0 && rand() < 0.1 ? 66 : 1;
   events.push({ type: 'gift', msgId: id(), ts, roomId: opts.roomId, user: u, tier, count, randomSkinGift: tier === 4 });
  } else if (r < opts.giftRatio + 0.6) {
   events.push({ type: 'like', msgId: id(), ts, roomId: opts.roomId, user: u, count: 1 + Math.floor(rand() * 10) });
  } else {
   const text = SLOGANS[Math.floor(rand() * SLOGANS.length)] ?? '冲！';
   events.push({ type: 'comment', msgId: id(), ts, roomId: opts.roomId, user: u, text });
```

```
    }
   }
   return events.sort((a, b) => a.ts - b.ts || a.msgId.localeCompare(b.msgId));
}
// packages/shared/src/text-filter.ts
/**
 * 词表过滤：命中检测、文本打码、昵称打码。纯函数、零依赖（客户端同步使用）。
 */
import { WORD_CATEGORIES, type WordCategory } from './wordlist-data';
export interface BannedHit {
  term: string;
  category: string;
  index: number;
  /** 命中的原文长度（UTF-16）。 */
  length: number;
}
interface CompiledList {
  zh: { term: string; category: string }[];
  en: { re: RegExp; term: string; category: string }[];
}
function escapeRegExp(s: string): string {
  return s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}
export function compileWordlist(categories: WordCategory[] = WORD_CATEGORIES): CompiledList {
  const zh: CompiledList['zh'] = [];
  const en: CompiledList['en'] = [];
  for (const c of categories) {
    for (const term of c.zh) zh.push({ term, category: c.id });
    for (const term of c.en) {
      const body = escapeRegExp(term).replace(/ /g, '[\\s_-]?');
      en.push({ re: new RegExp(`(?<![A-Za-z])${body}(?![A-Za-z])`, 'gi'), term, category: c.id });
    }
  }
  return { zh, en };
}
const DEFAULT_LIST = compileWordlist();
/** 找出文本中所有禁用词命中（按出现位置排序）。 */
export function findBanned(text: string, list: CompiledList = DEFAULT_LIST): BannedHit[] {
  const hits: BannedHit[] = [];
  for (const { term, category } of list.zh) {
    let from = 0;
    for (;;) {
      const i = text.indexOf(term, from);
      if (i < 0) break;
      hits.push({ term, category, index: i, length: term.length });
      from = i + term.length;
    }
  }
  for (const { re, term, category } of list.en) {
    re.lastIndex = 0;
```

```
    for (let m = re.exec(text); m; m = re.exec(text)) {
      hits.push({ term, category, index: m.index, length: m[0].length });
      if (m[0].length === 0) re.lastIndex++;
    }
  }
  return hits.sort((a, b) => a.index - b.index);
}
/** 把命中的禁用词逐字替换为 *。 */
export function maskText(text: string, list: CompiledList = DEFAULT_LIST): string {
  const chars = Array.from(text);
  // 以 UTF-16 下标定位，再映射到字符数组
  const offsets: number[] = [];
  let pos = 0;
  for (const ch of chars) {
    offsets.push(pos);
    pos += ch.length;
  }
  const masked = new Array<boolean>(chars.length).fill(false);
  for (const hit of findBanned(text, list)) {
    const end = hit.index + hit.length;
    chars.forEach((_, i) => {
      const o = offsets[i] ?? 0;
      if (o >= hit.index && o < end) masked[i] = true;
    });
  }
  return chars.map((ch, i) => (masked[i] ? '*' : ch)).join('');
}
export const NICK_MAX_CHARS = 10;
export const NICK_FALLBACK = '水友';
/**
 * 昵称上屏前处理：命中禁用词 → 保留命中前最多 2 个字 + "**"；过长截断。
 * 原始昵称不得出现在任何展示路径。
 */
export function maskNick(nick: string, list: CompiledList = DEFAULT_LIST): string {
  const trimmed = nick.trim();
  if (trimmed.length === 0) return NICK_FALLBACK;
  if (findBanned(trimmed, list).length > 0) {
    const masked = maskText(trimmed, list);
    const prefix = Array.from(masked.split('*')[0] ?? '').slice(0, 2).join('');
    return `${prefix || NICK_FALLBACK}**`;
  }
  const chars = Array.from(trimmed);
  return chars.length > NICK_MAX_CHARS ? `${chars.slice(0, NICK_MAX_CHARS).join('')}…` : trimmed;
}
// packages/shared/src/wordlist-data.ts
/**
 * 外显禁用词表（单一数据源）。CI 扫描器、昵称/弹幕过滤、TTS 文案过滤共用。
 * 来源：方案 2.1（赌具/机制）、2.3（用词规避）、5.8 原则 1/7（金额与催氪句式）、Goal v1.2（内部代号）。
 * 追加新词即生效，无需改动扫描器或过滤器代码。
 */
```

```
export interface WordCategory {
 id: string;
 reason: string;
 /** 中文词：子串匹配。 */
 zh: string[];
 /** 英文词：按词边界、大小写不敏感匹配。 */
 en: string[];
}
export const WORD_CATEGORIES: WordCategory[] = [
 {
  id: 'concept',
  reason: '方案 2.3 概念类（平台视为游戏化包装）',
  zh: ['游戏', '玩家', '背包', '血条', '赛季', '排位', '段位'],
  en: ['BUFF', 'CD', 'PVP', 'MOBA'],
 },
 {
  id: 'random',
  reason: '方案 2.1/2.3 随机类（抽奖/盲盒外显禁止）',
  zh: ['抽奖', '盲盒', '礼盒', '开箱', '扭蛋', '抽卡'],
  en: ['lucky draw', 'luckydraw', 'luckydrawer', 'gacha', 'loot box', 'lootbox'],
 },
 {
  id: 'combat',
  reason: '方案 2.3 战斗类',
  zh: ['连杀', '吃鸡'],
  en: ['DPS', 'HP'],
 },
 {
  id: 'gambling',
  reason: '方案 2.0/2.1 涉赌机制与赌具',
  zh: ['赌', '博彩', '押注', '下注', '竞猜', '老虎机', '推币机', '打鱼机', '捕鱼机', '上分器', '赛马', '百家乐', '扎金花', '21点', '梭哈'],
  en: [],
 },
 {
  id: 'money',
  reason: '方案 5.8 原则 1 零金额显示 / 禁消费字样',
  zh: ['充值', '消费', '花钱', '价格', '返现', '提现', '兑现', '抖币'],
  en: [],
 },
 {
  id: 'pressure',
  reason: '方案 5.8 原则 7 催氪红线句式',
  zh: ['再送', '快送', '刷起来', '最后一个名额', '马上下架', '榜一都比你', '日赚', '躺赚'],
  en: [],
 },
];
// packages/core/src/index.ts
export * from './types';
export { createRoomState, step, phaseEndsAt, balanceWeight, multiplier, lossBonus } from './kernel';
export type { StepResult } from './kernel';
```

```
export { toSnapshot, topRanks, topCheers, buildSettleResult } from './snapshot';
export { snapshot } from './view';
// packages/core/src/kernel.ts
/**
 * 对局内核：step(state, input, config) -> { state, effects }。
 *
 * - 确定性：无 IO、不读实时钟、不用 Math.random()；时间只来自输入时间戳。
 * - 线性所有权：传入的 state 被就地推进并返回（高频事件下避免整份拷贝）；
 *   调用方不得在 step 之后继续使用旧引用做比较。回放 = 从初始状态重放同一输入序列。
 * - 唯一胜负条件：任一方进度先达 winProgress。计时耗尽进入终局潮汐，不引入第二条件。
 */
import {
  SIDES,
  hashString,
  mulberry32,
  otherSide,
  pickWeighted,
  type CommentEvent,
  type GameConfig,
  type GiftEvent,
  type LikeEvent,
  type LiveUser,
  type SideId,
  type Tier,
  type TierEffect,
} from '@jinli/shared';
import type { CoreInput, CounterKind, Effect, RoomState, SideState, UserState } from './types';
import { buildSettleResult } from './snapshot';
/** 单个积分步长上限（持续效果与阶段边界的时间分辨率）。 */
const MAX_CHUNK_MS = 250;
/** 单条礼物消息逐件结算的上限（超出部分只计助力值，防异常数据卡死内核）。 */
const MAX_ITEMS_PER_GIFT = 200;
type GainSource = 'fry' | 'paid' | 'free' | 'overtime';
function newSide(lossStreak = 0): SideState {
  return {
    progress: 0,
    members: 0,
    power: 0,
    charge: 0,
    giftChargeThisCycle: 0,
    accelUntil: 0,
    accelBoost: 0,
    strengthenUntil: 0,
    strengthenBoost: 0,
    shieldUntil: 0,
    eliteUntil: 0,
    eliteSuppress: 0,
    dragonUntil: 0,
    dragonSuppress: 0,
    lossStreak,
```

```
    carry: 0,
    lastContributor: null,
   };
}
/** 开局：创建房间并进入第一局蓄势阶段。 */
export function createRoomState(roomId: string, ts: number): RoomState {
  return {
    roomId,
    matchNo: 1,
    phase: 'charge',
    phaseStartedAt: ts,
    matchStartedAt: ts,
    now: ts,
    sides: { A: newSide(), B: newSide() },
    users: {},
    userSeq: 0,
    flows: [],
    comebackSide: null,
    comebackAnnounced: null,
    dragonsThisMatch: 0,
    result: null,
  };
}
export interface StepResult {
  state: RoomState;
  effects: Effect[];
}
export function step(state: RoomState, input: CoreInput, config: GameConfig): StepResult {
  const effects: Effect[] = [];
  advance(state, Math.max(input.ts, state.now), config, effects);
  switch (input.type) {
    case 'tick':
      break;
    case 'comment':
      onComment(state, input, config, effects);
      break;
    case 'like':
      onLike(state, input, config, effects);
      break;
    case 'gift':
      onGift(state, input, config, effects);
      break;
    case 'join':
      effects.push({ kind: 'enter', at: state.now, nick: input.user.nick });
      break;
    case 'follow':
      effects.push({ kind: 'follow', at: state.now, nick: input.user.nick });
      break;
  }
  if (input.type !== 'tick') checkWin(state, config, effects);
```

```
  return { state, effects };
}
/** 当前阶段结束时间；终局潮汐无固定结束（null）。 */
export function phaseEndsAt(state: RoomState, config: GameConfig): number | null {
  const p = config.phases;
  switch (state.phase) {
    case 'charge': {
      const members = state.sides.A.members + state.sides.B.members;
      const dur = members < p.sparseThreshold ? p.chargeMsSparse : p.chargeMs;
      return state.phaseStartedAt + dur;
    }
    case 'clash':
      return state.phaseStartedAt + p.clashMs;
    case 'fortune':
      return state.phaseStartedAt + p.fortuneMs;
    case 'overtime':
      return null;
    case 'settle':
      return state.phaseStartedAt + p.settleMs;
  }
}
// ——————————————————————————————————————— 时间推进 ———————————————————————————————————————
function advance(state: RoomState, to: number, config: GameConfig, effects: Effect[]): void {
  while (state.now < to) {
    const boundary = phaseEndsAt(state, config) ?? Number.POSITIVE_INFINITY;
    const chunkEnd = Math.min(to, boundary, state.now + MAX_CHUNK_MS);
    const t0 = state.now;
    state.now = chunkEnd;
    if (state.phase !== 'settle') {
      integrate(state, t0, chunkEnd - t0, config);
      checkWin(state, config, effects);
    }
    const end = phaseEndsAt(state, config);
    if (end !== null && state.now >= end) transition(state, config, effects);
  }
}
/** 持续性推进：鱼苗/全队流 + 终局潮汐自动推进。 */
function integrate(state: RoomState, t0: number, dt: number, config: GameConfig): void {
  if (dt <= 0) return;
  const kept = [];
  for (const f of state.flows) {
    const amount = Math.min(f.remaining, f.perMs * dt);
    f.remaining -= amount;
    gain(state, f.side, amount, f.kind === 'fry' ? 'fry' : 'paid', f.userId, config, t0);
    if (f.remaining > 1e-9) kept.push(f);
  }
  state.flows = kept;
  if (state.phase === 'overtime') {
    const amount = (config.overtimeRatePerSec * dt) / 1000;
    for (const side of SIDES) gain(state, side, amount, 'overtime', null, config, t0);
```

```
  }
}
function transition(state: RoomState, config: GameConfig, effects: Effect[]): void {
  const next: Record<RoomState['phase'], RoomState['phase'] | 'new-match'> = {
    charge: 'clash',
    clash: 'fortune',
    fortune: 'overtime',
    overtime: 'overtime',
    settle: 'new-match',
  };
  const target = next[state.phase];
  if (target === 'new-match') {
    startNextMatch(state, config, effects);
    return;
  }
  state.phase = target;
  state.phaseStartedAt = state.now;
  effects.push({ kind: 'phase', at: state.now, phase: target, matchNo: state.matchNo, endsAt: phaseEndsAt(state, config) });
}
function startNextMatch(state: RoomState, config: GameConfig, effects: Effect[]): void {
  const carry = { A: state.sides.A.carry, B: state.sides.B.carry };
  state.matchNo += 1;
  state.phase = 'charge';
  state.phaseStartedAt = state.now;
  state.matchStartedAt = state.now;
  state.sides = { A: newSide(state.sides.A.lossStreak), B: newSide(state.sides.B.lossStreak) };
  state.users = {};
  state.userSeq = 0;
  state.flows = [];
  state.comebackSide = null;
  state.comebackAnnounced = null;
  state.dragonsThisMatch = 0;
  state.result = null;
  effects.push({ kind: 'phase', at: state.now, phase: 'charge', matchNo: state.matchNo, endsAt: phaseEndsAt(state, config) });
  for (const side of SIDES) if (carry[side] > 0) gain(state, side, carry[side], 'paid', null, config, state.now);
}
// ──────────────────────────────────────── 推进与乘数 ────────────────────────────────────────
/** 人少方产出加权：w = clamp((N多/N少)^exp, 1, max)。 */
export function balanceWeight(state: RoomState, side: SideId, config: GameConfig): number {
  const mine = state.sides[side].members;
  const theirs = state.sides[otherSide(side)].members;
  if (mine === 0 || mine >= theirs) return 1;
  const w = Math.pow(theirs / mine, config.balance.exponent);
  return Math.min(Math.max(w, 1), config.balance.maxWeight);
}
export function lossBonus(sideState: SideState, config: GameConfig): number {
  return Math.min(sideState.lossStreak * config.balance.lossStreakStep, config.balance.lossStreakCap);
}
export function multiplier(state: RoomState, side: SideId, source: GainSource, config: GameConfig, t: number): number {
  if (source === 'overtime') return 1;
```

```
  const s = state.sides[side];
  const o = state.sides[otherSide(side)];
  let m = state.phase === 'fortune' ? config.fortuneMultiplier : 1;
  m *= balanceWeight(state, side, config);
  if (state.comebackSide === side) m *= config.balance.comebackBoost;
  m *= 1 + lossBonus(s, config);
  const accelOn = s.accelUntil > t;
  let boost = 0;
  if (accelOn) boost += s.accelBoost;
  if (s.strengthenUntil > t) boost += s.strengthenBoost;
  m *= 1 + boost;
  // 加速克爆发：加速中所受压制减半
  const counter = accelOn ? config.counterFactor : 1;
  if (o.dragonUntil > t) m *= 1 - o.dragonSuppress * counter;
  if (source === 'fry' && o.eliteUntil > t) m *= 1 - o.eliteSuppress * counter;
  return Math.max(m, 0);
}
function gain(
  state: RoomState,
  side: SideId,
  base: number,
  source: GainSource,
  userId: string | null,
  config: GameConfig,
  t: number,
): number {
  if (base <= 0) return 0;
  if (state.phase === 'settle') {
    state.sides[side].carry += base;
    return 0;
  }
  const s = state.sides[side];
  const ceil = phaseCeil(state, config);
  const amount = Math.min(base * multiplier(state, side, source, config, t), Math.max(ceil - s.progress, 0));
  s.progress += amount;
  if (userId !== null) {
    const u = state.users[userId];
    if (u) u.progress += amount;
    s.lastContributor = userId;
  }
  updateComeback(state, config);
  return amount;
}
/** 当前阶段的进度上限（当量）。鸿运/终局潮汐不设上限（可抵达 100%）。 */
function phaseCeil(state: RoomState, config: GameConfig): number {
  switch (state.phase) {
    case 'charge':
      return config.winProgress * config.progressCeil.charge;
    case 'clash':
      return config.winProgress * config.progressCeil.clash;
```

```
    default:
      return Number.POSITIVE_INFINITY;
  }
}
function updateComeback(state: RoomState, config: GameConfig): void {
  const diff = (state.sides.A.progress - state.sides.B.progress) / config.winProgress;
  const trailing: SideId = diff > 0 ? 'B' : 'A';
  const gap = Math.abs(diff);
  if (state.comebackSide === null) {
    if (gap >= config.balance.comebackOn) state.comebackSide = trailing;
  } else if (gap < config.balance.comebackOff) {
    state.comebackSide = null;
  } else if (trailing !== state.comebackSide && gap >= config.balance.comebackOn) {
    state.comebackSide = trailing;
  }
}
/** 唯一胜负条件：进度达胜负线。同帧双方越线 → 进度高者胜，完全相等 → 平局。 */
function checkWin(state: RoomState, config: GameConfig, effects: Effect[]): void {
  if (state.phase === 'settle') {
    return;
  }
  const crossed = SIDES.filter((s) => state.sides[s].progress >= config.winProgress);
  if (crossed.length === 0) {
    updateComeback(state, config);
    trackComebackEffect(state, effects);
    return;
  }
  let winner: SideId | 'draw';
  if (crossed.length === 1) winner = crossed[0] as SideId;
  else {
    const a = state.sides.A.progress;
    const b = state.sides.B.progress;
    winner = a === b ? 'draw' : a > b ? 'A' : 'B';
  }
  finish(state, winner, config, effects);
}
function trackComebackEffect(state: RoomState, effects: Effect[]): void {
  if (state.comebackAnnounced !== state.comebackSide) {
    state.comebackAnnounced = state.comebackSide;
    effects.push({ kind: 'comeback', at: state.now, side: state.comebackSide });
  }
}
function finish(state: RoomState, winner: SideId | 'draw', config: GameConfig, effects: Effect[]): void {
  const reason = state.phase === 'overtime' ? 'overtime' : 'reach';
  for (const side of SIDES) {
    const s = state.sides[side];
    s.progress = Math.min(s.progress, config.winProgress);
  }
  state.result = buildSettleResult(state, winner, reason, config);
  if (winner !== 'draw') {
```

```
    state.sides[winner].lossStreak = 0;
    state.sides[otherSide(winner)].lossStreak += 1;
  }
  state.phase = 'settle';
  state.phaseStartedAt = state.now;
  state.flows = [];
  effects.push({ kind: 'phase', at: state.now, phase: 'settle', matchNo: state.matchNo, endsAt: phaseEndsAt(state, config) });
  effects.push({ kind: 'settle', at: state.now, result: state.result });
}
// ──────────────────────────────────────────────────────────── 用户与阵营 ────────────────────────────────────────────────────────
function smallerSide(state: RoomState): SideId {
  const a = state.sides.A.members;
  const b = state.sides.B.members;
  if (a !== b) return a < b ? 'A' : 'B';
  return state.sides.A.progress <= state.sides.B.progress ? 'A' : 'B';
}
function ensureUser(state: RoomState, user: LiveUser, preferred: SideId | null, effects: Effect[]): UserState {
  const existing = state.users[user.id];
  if (existing) {
    existing.nick = user.nick;
    return existing;
  }
  const side = preferred ?? smallerSide(state);
  const u: UserState = {
    id: user.id,
    nick: user.nick,
    side,
    seq: state.userSeq++,
    power: 0,
    progress: 0,
    cheers: 0,
    bigGifts: 0,
    dragons: 0,
    lastCommentAt: Number.NEGATIVE_INFINITY,
    likeWindowStart: Number.NEGATIVE_INFINITY,
    likeWindowCount: 0,
  };
  state.users[user.id] = u;
  state.sides[side].members += 1;
  effects.push({ kind: 'join', at: state.now, side, nick: user.nick });
  return u;
}
function addCharge(state: RoomState, side: SideId, amount: number, fromGift: boolean, config: GameConfig, effects: Effect[]): void {
  if (state.phase === 'settle' || amount <= 0) return;
  const s = state.sides[side];
  let add = amount;
  if (fromGift) {
    const room = config.charge.cap * config.charge.giftShareCap - s.giftChargeThisCycle;
    add = Math.min(add, Math.max(room, 0));
    s.giftChargeThisCycle += add;
```

```
  }
  s.charge += add;
  if (s.charge >= config.charge.cap) {
    s.charge = 0;
    s.giftChargeThisCycle = 0;
    gain(state, side, config.charge.burstProgress, 'free', null, config, state.now);
    effects.push({ kind: 'surge', at: state.now, side });
  }
}
function sideKeyword(text: string, config: GameConfig): SideId | null {
  const t = text.trim();
  for (const side of SIDES) {
    if (config.sides[side].joinKeywords.some((k) => t === k || t.startsWith(k))) return side;
  }
  return null;
}
// ──────────────────────────────────────────── 免费互动 ────────────────────────────────────────────
function onComment(state: RoomState, ev: CommentEvent, config: GameConfig, effects: Effect[]): void {
  if (state.phase === 'settle') {
    effects.push({ kind: 'comment', at: state.now, side: null, nick: ev.user.nick, text: ev.text });
    return;
  }
  const u = ensureUser(state, ev.user, sideKeyword(ev.text, config), effects);
  effects.push({ kind: 'comment', at: state.now, side: u.side, nick: u.nick, text: ev.text });
  if (state.now - u.lastCommentAt < config.free.commentCooldownMs) return;
  u.lastCommentAt = state.now;
  u.cheers += 1;
  u.power += config.free.commentProgress;
  state.sides[u.side].power += config.free.commentProgress;
  gain(state, u.side, config.free.commentProgress, 'free', u.id, config, state.now);
  addCharge(state, u.side, config.free.commentCharge, false, config, effects);
}
function onLike(state: RoomState, ev: LikeEvent, config: GameConfig, effects: Effect[]): void {
  if (state.phase === 'settle') {
    effects.push({ kind: 'like', at: state.now, side: null, nick: ev.user.nick, count: ev.count });
    return;
  }
  const u = ensureUser(state, ev.user, null, effects);
  if (state.now - u.likeWindowStart >= 1000) {
    u.likeWindowStart = state.now;
    u.likeWindowCount = 0;
  }
  const allowed = Math.max(0, Math.min(ev.count, config.free.likePerUserPerSecCap - u.likeWindowCount));
  u.likeWindowCount += allowed;
  effects.push({ kind: 'like', at: state.now, side: u.side, nick: u.nick, count: ev.count });
  if (allowed === 0) return;
  const p = allowed * config.free.likeProgress;
  u.cheers += allowed;
  u.power += p;
  state.sides[u.side].power += p;
```

```
 gain(state, u.side, p, 'free', u.id, config, state.now);
 addCharge(state, u.side, allowed * config.free.likeCharge, false, config, effects);
}
// ──────────────────────────────────────────────── 礼物 ────────────────────────────────────────────────
function onGift(state: RoomState, ev: GiftEvent, config: GameConfig, effects: Effect[]): void {
 const count = Math.max(1, Math.floor(ev.count));
 const powerRule = config.tiers[ev.tier];
 const settling = state.phase === 'settle';
 // 结算期：沿用本局阵营（无则人少方），加持兑现到下一局
 const u = settling ? state.users[ev.user.id] ?? null : ensureUser(state, ev.user, null, effects);
 const side = u ? u.side : smallerSide(state);
 const t = state.now;
 const m = multiplier(state, side, 'paid', config, t);
 let effectTier: Tier = ev.tier;
 let nominal = 0;
 const skins: string[] = [];
 let rare = false;
 const items = Math.min(count, MAX_ITEMS_PER_GIFT);
 if (u) {
  u.power += powerRule.power * count;
 }
 state.sides[side].power += powerRule.power * count;
 for (let i = 0; i < items; i++) {
  let tier: Tier = ev.tier;
  // 模式 A 专属：只有指定随机外观款能召唤精英锦鲤
  if (tier === 4 && !ev.randomSkinGift) tier = config.randomSkin.fallbackTier;
  if (tier === 6 && !settling) {
   const full = state.dragonsThisMatch >= config.dragon.perMatch || (u !== null && u.dragons >= config.dragon.perUser);
   if (full) {
    if (i === 0) effects.push({ kind: 'dragon-full', at: t, side, nick: ev.user.nick });
    tier = config.dragon.fallbackTier;
   } else {
    state.dragonsThisMatch += 1;
    if (u) u.dragons += 1;
   }
  }
  effectTier = tier;
  const rule = config.tiers[tier];
  let personal = 1;
  if (ev.tier >= config.whale.fromTier && u) {
   u.bigGifts += 1;
   if (u.bigGifts >= config.whale.fromNth) personal = config.whale.personalShare;
  }
  const p = rule.progress;
  if (settling) {
   state.sides[side].carry += p;
  } else if (rule.delivery === 'flow' && rule.flowMs > 0) {
   addFlow(state, side, 'fry', p * personal, rule.flowMs, u?.id ?? null);
  } else {
   gain(state, side, p * personal, 'paid', u?.id ?? null, config, t);
```

```
  }
  if (!settling && personal < 1) addFlow(state, side, 'team', p * (1 - personal), config.whale.teamFlowMs, null);
  nominal += p * m;
  if (!settling) {
   const skin = applyEffect(state, side, rule.effect, ev, i, config, effects);
   if (skin) {
    skins.push(skin.label);
    rare = rare || skin.rare;
   }
   addCharge(state, side, powerRule.power * config.charge.giftChargePerPower, true, config, effects);
  }
 }
 effects.push({
  kind: 'gift',
  at: t,
  side,
  nick: ev.user.nick,
  tier: ev.tier,
  effectTier,
  count,
  label: config.tiers[effectTier].label,
  progressPct: Math.round((nominal / config.winProgress) * 1000) / 10,
  skins,
  rare,
  carried: settling,
 });
 if (!settling && config.tiers[effectTier].effect.kind === 'leap') {
  effects.push({ kind: 'leap', at: t, side, nick: ev.user.nick });
 }
}
function addFlow(state: RoomState, side: SideId, kind: 'fry' | 'team', total: number, durationMs: number, userId: string | null): void {
 if (total <= 0) return;
 state.flows.push({ side, kind, perMs: total / durationMs, remaining: total, userId, items: kind === 'fry' ? 1 : 0 });
}
function counter(effects: Effect[], at: number, side: SideId, kind: CounterKind): void {
 effects.push({ kind: 'counter', at, side, counter: kind });
}
function applyEffect(
 state: RoomState,
 side: SideId,
 effect: TierEffect,
 ev: GiftEvent,
 index: number,
 config: GameConfig,
 effects: Effect[],
): { label: string; rare: boolean } | null {
 const t = state.now;
 const s = state.sides[side];
 const opp = otherSide(side);
 const o = state.sides[opp];
```

```
 switch (effect.kind) {
  case 'fry':
  case 'leap':
   return null;
  case 'accel': {
   let boost = effect.boost;
   // 护盾克加速：对方护盾在场时本方加速减半
   if (o.shieldUntil > t) {
    boost *= config.counterFactor;
    counter(effects, t, opp, 'shield-vs-accel');
   }
   s.accelUntil = t + effect.durationMs;
   s.accelBoost = boost;
   return null;
  }
  case 'shield':
   s.shieldUntil = t + effect.durationMs;
   return null;
  case 'strengthen':
   s.strengthenUntil = t + effect.durationMs;
   s.strengthenBoost = effect.boost;
   return null;
  case 'elite': {
   s.eliteUntil = t + effect.durationMs;
   s.eliteSuppress = burstAgainst(state, side, effect.suppressFry, config, effects);
   if (!ev.randomSkinGift) return null;
   const skinRules = config.randomSkin.skins;
   const r = mulberry32(hashString(`${ev.msgId}:${index}`))();
   const skin = skinRules[pickWeighted(skinRules.map((k) => k.weight), r)];
   return skin ? { label: skin.label, rare: skin.rare } : null;
  }
  case 'dragon':
   s.dragonUntil = t + effect.durationMs;
   s.dragonSuppress = burstAgainst(state, side, effect.suppress, config, effects);
   return null;
 }
}
/** 爆发类压制落到对方：爆发克护盾（护盾破碎，吸收一半）；对方加速中则在乘数里再减半。 */
function burstAgainst(state: RoomState, side: SideId, suppress: number, config: GameConfig, effects: Effect[]): number {
 const t = state.now;
 const opp = otherSide(side);
 const o = state.sides[opp];
 let value = suppress;
 if (o.shieldUntil > t) {
  o.shieldUntil = t;
  value *= config.counterFactor;
  counter(effects, t, side, 'burst-vs-shield');
 }
 if (o.accelUntil > t) counter(effects, t, opp, 'accel-vs-burst');
 return value;
```

```
}
// packages/core/src/snapshot.ts
/**
 * 对外输出：结算结果与房间快照。只含助力值、进度比例与展示信息，无金额字段。
 */
import { otherSide, type GameConfig, type SideId } from '@jinli/shared';
import type { CheerEntry, RankEntry, RoomState, SettleResult, Snapshot, SideView, UserState } from './types';
function byPower(a: UserState, b: UserState): number {
  return b.power - a.power || a.seq - b.seq;
}
function byCheers(a: UserState, b: UserState): number {
  return b.cheers - a.cheers || a.seq - b.seq;
}
function rankEntry(state: RoomState, u: UserState): RankEntry {
  const sideProgress = state.sides[u.side].progress;
  const pct = sideProgress > 0 ? (u.progress / sideProgress) * 100 : 0;
  return { nick: u.nick, side: u.side, power: Math.round(u.power), contributionPct: Math.round(pct * 10) / 10 };
}
export function topRanks(state: RoomState, n: number): RankEntry[] {
  return Object.values(state.users).sort(byPower).slice(0, n).map((u) => rankEntry(state, u));
}
export function topCheers(state: RoomState, n: number): CheerEntry[] {
  return Object.values(state.users)
    .filter((u) => u.cheers > 0)
    .sort(byCheers)
    .slice(0, n)
    .map((u) => ({ nick: u.nick, side: u.side, cheers: u.cheers }));
}
export function buildSettleResult(
  state: RoomState,
  winner: SideId | 'draw',
  reason: 'reach' | 'overtime',
  config: GameConfig,
): SettleResult {
  let keyPlayer: SettleResult['keyPlayer'] = null;
  if (winner !== 'draw') {
    const id = state.sides[winner].lastContributor;
    const u = id !== null ? state.users[id] : undefined;
    if (u) keyPlayer = { nick: u.nick, side: u.side };
  }
  const base = config.settle.starsBase;
  return {
    matchNo: state.matchNo,
    winner,
    reason,
    durationMs: state.now - state.matchStartedAt,
    keyPlayer,
    ranks: topRanks(state, config.settle.topN),
    cheers: topCheers(state, 3),
    participants: Object.keys(state.users).length,
```

```
    stars: { winnerEach: base + config.settle.starsWinBonus, loserEach: winner === 'draw' ? base + config.settle.starsWinBonus : base },
  };
}
function sideView(state: RoomState, side: SideId, config: GameConfig): SideView {
  const s = state.sides[side];
  const o = state.sides[otherSide(side)];
  const t = state.now;
  return {
    name: config.sides[side].name,
    progress: Math.min(s.progress / config.winProgress, 1),
    members: s.members,
    power: Math.round(s.power),
    charge: Math.min(s.charge / config.charge.cap, 1),
    accel: s.accelUntil > t,
    strengthen: s.strengthenUntil > t,
    shield: s.shieldUntil > t,
    elite: s.eliteUntil > t,
    dragon: s.dragonUntil > t,
    suppressed: o.dragonUntil > t,
    comeback: state.comebackSide === side,
    fry: state.flows.filter((f) => f.side === side && f.kind === 'fry').length,
    lossBonus: Math.min(s.lossStreak * config.balance.lossStreakStep, config.balance.lossStreakCap),
  };
}
export function toSnapshot(state: RoomState, config: GameConfig, phaseEndsAt: number | null): Snapshot {
  return {
    roomId: state.roomId,
    matchNo: state.matchNo,
    phase: state.phase,
    now: state.now,
    phaseEndsAt,
    sides: { A: sideView(state, 'A', config), B: sideView(state, 'B', config) },
    top: topRanks(state, 5),
    cheers: topCheers(state, 3),
    result: state.result,
  };
}
// packages/core/src/types.ts
/**
 * core 对局内核的状态、输入与输出类型。
 * 输出（Effect / Snapshot / SettleResult）只含助力值、进度比例与展示信息，绝无金额字段。
 */
import type { LiveEvent, SideId, Tier } from '@jinli/shared';
export type Phase = 'charge' | 'clash' | 'fortune' | 'overtime' | 'settle';
/** 时钟推进输入（由外壳按固定节拍注入，携带显式时间戳）。 */
export interface TickInput {
  type: 'tick';
  ts: number;
}
export type CoreInput = LiveEvent | TickInput;
```

```
/** 持续兑现的推进（鱼苗单位 / 防鲸差额转全队）。 */
export interface Flow {
  side: SideId;
  kind: 'fry' | 'team';
  perMs: number;
  remaining: number;
  userId: string | null;
  items: number;
}
export interface SideState {
  /** 进度（当量，0~winProgress；结算判定瞬间可短暂越线）。 */
  progress: number;
  members: number;
  /** 本方累计助力值。 */
  power: number;
  charge: number;
  giftChargeThisCycle: number;
  accelUntil: number;
  accelBoost: number;
  strengthenUntil: number;
  strengthenBoost: number;
  shieldUntil: number;
  /** 本方精英锦鲤：压制对方鱼苗产出。 */
  eliteUntil: number;
  eliteSuppress: number;
  /** 本方神龙：压制对方全体推进。 */
  dragonUntil: number;
  dragonSuppress: number;
  /** 连败场次（跨局保留）。 */
  lossStreak: number;
  /** 本局结算期送出的加持，下一局开局兑现。 */
  carry: number;
  lastContributor: string | null;
}
export interface UserState {
  id: string;
  nick: string;
  side: SideId;
  seq: number;
  power: number;
  progress: number;
  cheers: number;
  bigGifts: number;
  dragons: number;
  lastCommentAt: number;
  likeWindowStart: number;
  likeWindowCount: number;
}
export interface RankEntry {
  nick: string;
```

```
  side: SideId;
  power: number;
  /** 个人推进占本方总进度百分比（个人归因，方案 5.7.A）。 */
  contributionPct: number;
}
export interface CheerEntry {
  nick: string;
  side: SideId;
  cheers: number;
}
export interface SettleResult {
  matchNo: number;
  winner: SideId | 'draw';
  reason: 'reach' | 'overtime';
  durationMs: number;
  /** 关键先生：打出越线一击者。 */
  keyPlayer: { nick: string; side: SideId } | null;
  ranks: RankEntry[];
  cheers: CheerEntry[];
  participants: number;
  /** 幸运星确定性发放（模式 B 免费资源）：胜方/平局每人 base+bonus，败方每人 base。 */
  stars: { winnerEach: number; loserEach: number };
}
export interface RoomState {
  roomId: string;
  matchNo: number;
  phase: Phase;
  phaseStartedAt: number;
  matchStartedAt: number;
  now: number;
  sides: Record<SideId, SideState>;
  users: Record<string, UserState>;
  userSeq: number;
  flows: Flow[];
  comebackSide: SideId | null;
  /** 已对外播报的逆流状态（去抖）。 */
  comebackAnnounced: SideId | null;
  dragonsThisMatch: number;
  result: SettleResult | null;
}
export type CounterKind = 'accel-vs-burst' | 'shield-vs-accel' | 'burst-vs-shield';
export type Effect =
  | { kind: 'phase'; at: number; phase: Phase; matchNo: number; endsAt: number | null }
  | { kind: 'join'; at: number; side: SideId; nick: string }
  | { kind: 'enter'; at: number; nick: string }
  | { kind: 'follow'; at: number; nick: string }
  | { kind: 'comment'; at: number; side: SideId | null; nick: string; text: string }
  | { kind: 'like'; at: number; side: SideId | null; nick: string; count: number }
  | {
      kind: 'gift';
```

```
    at: number;
    side: SideId;
    nick: string;
    tier: Tier;
    effectTier: Tier;
    count: number;
    label: string;
    /** 本次送礼推进本方进度的百分比（个人归因）。 */
    progressPct: number;
    skins: string[];
    rare: boolean;
    /** 结算期送出：兑现到下一局开局。 */
    carried: boolean;
  }
 | { kind: 'surge'; at: number; side: SideId }
 | { kind: 'counter'; at: number; side: SideId; counter: CounterKind }
 | { kind: 'dragon-full'; at: number; side: SideId; nick: string }
 | { kind: 'comeback'; at: number; side: SideId | null }
 | { kind: 'leap'; at: number; side: SideId; nick: string }
 | { kind: 'settle'; at: number; result: SettleResult };
export interface SideView {
 name: string;
 /** 进度比例 0~1。 */
 progress: number;
 members: number;
 power: number;
 /** 集体充能比例 0~1。 */
 charge: number;
 accel: boolean;
 strengthen: boolean;
 shield: boolean;
 elite: boolean;
 dragon: boolean;
 /** 正被对方神龙压制。 */
 suppressed: boolean;
 comeback: boolean;
 fry: number;
 lossBonus: number;
}
export interface Snapshot {
 roomId: string;
 matchNo: number;
 phase: Phase;
 now: number;
 phaseEndsAt: number | null;
 sides: Record<SideId, SideView>;
 top: RankEntry[];
 cheers: CheerEntry[];
 result: SettleResult | null;
}
```

```
// packages/core/src/view.ts
import type { GameConfig } from '@jinli/shared';
import { phaseEndsAt } from './kernel';
import { toSnapshot } from './snapshot';
import type { RoomState, Snapshot } from './types';
/** 生成当前房间快照（供广播/渲染）。 */
export function snapshot(state: RoomState, config: GameConfig): Snapshot {
 return toSnapshot(state, config, phaseEndsAt(state, config));
}
// packages/core/test/kernel.test.ts
import { describe, expect, it } from 'vitest';
import { DEFAULT_SIM, generateSimEvents, makeConfig, type GiftEvent, type LiveEvent, type SideId, type Tier } from '@jinli/shared';
import { balanceWeight, createRoomState, snapshot, step, type Effect, type RoomState } from '../src';
const cfg = makeConfig();
let seq = 0;
const id = () => `m${seq++}`;
function run(state: RoomState, inputs: Parameters<typeof step>[1][]): Effect[] {
 const out: Effect[] = [];
 for (const i of inputs) out.push(...step(state, i, cfg).effects);
 return out;
}
function tickTo(state: RoomState, ts: number, every = 200): Effect[] {
 const out: Effect[] = [];
 for (let t = state.now + every; t <= ts; t += every) out.push(...step(state, { type: 'tick', ts: t }, cfg).effects);
 return out;
}
function join(state: RoomState, userId: string, side: SideId, ts = state.now): Effect[] {
 return run(state, [{ type: 'comment', msgId: id(), ts, roomId: 'r', user: { id: userId, nick: userId }, text: cfg.sides[side].name }]);
}
function gift(state: RoomState, userId: string, tier: Tier, opts: Partial<GiftEvent> = {}): Effect[] {
 return run(state, [
  { type: 'gift', msgId: id(), ts: state.now, roomId: 'r', user: { id: userId, nick: userId }, tier, count: 1, randomSkinGift: tier === 4, ...opts },
 ]);
}
/** 进入攻防阶段的房间：A 方 1 人，B 方 0 人（乘数为 1）。 */
function clashRoom(): RoomState {
 const s = createRoomState('r', 0);
 join(s, 'a1', 'A');
 tickTo(s, cfg.phases.chargeMsSparse + 200);
 expect(s.phase).toBe('clash');
 s.sides.A.progress = 0;
 s.sides.A.power = 0;
 s.users.a1!.power = 0;
 return s;
}
describe('phases', () => {
 it('runs an empty room through all phases, ends via overtime as a draw, then starts match 2', () => {
  const s = createRoomState('r', 0);
  const phases: string[] = [];
  for (const e of tickTo(s, 2_000_000, 1000)) if (e.kind === 'phase') phases.push(`${e.matchNo}:${e.phase}`);
```

```
  expect(phases.slice(0, 6)).toEqual(['1:clash', '1:fortune', '1:overtime', '1:settle', '2:charge', '2:clash']);
  const settle = run(createRoomState('x', 0), []);
  expect(settle).toEqual([]);
 });
 it('shortens the charge phase when the room is sparse', () => {
  const sparse = createRoomState('r', 0);
  tickTo(sparse, cfg.phases.chargeMsSparse);
  expect(sparse.phase).toBe('clash');
  const busy = createRoomState('r', 0);
  for (let i = 0; i < cfg.phases.sparseThreshold; i++) join(busy, `u${i}`, i % 2 ? 'A' : 'B', 0);
  tickTo(busy, cfg.phases.chargeMsSparse + 1000);
  expect(busy.phase).toBe('charge');
  tickTo(busy, cfg.phases.chargeMs);
  expect(busy.phase).toBe('clash');
 });
 it('overtime draw is resolved by equal auto-progress and records reason', () => {
  const s = createRoomState('r', 0);
  const effects = tickTo(s, 1_000_000, 1000);
  const settle = effects.find((e) => e.kind === 'settle');
  expect(settle && settle.kind === 'settle' && settle.result).toMatchObject({ winner: 'draw', reason: 'overtime' });
 });
});
describe('tier economy', () => {
 const expected: Record<Tier, { power: number; progress: number }> = {
  0: { power: 25, progress: 25 },
  1: { power: 100, progress: 100 },
  2: { power: 260, progress: 260 },
  3: { power: 520, progress: 520 },
  4: { power: 1050, progress: 1000 },
  5: { power: 2600, progress: 1000 },
  6: { power: 5200, progress: 800 },
  7: { power: 13000, progress: 960 },
 };
 for (const tier of [0, 1, 2, 3, 4, 5, 6, 7] as Tier[]) {
  it(`T${tier} credits power and progress`, () => {
   const s = clashRoom();
   gift(s, 'a1', tier);
   // T2 自带加速、T5 自带强化，会放大后续流；鱼苗流在 60s 内兑现完毕
   tickTo(s, s.now + 61_000);
   expect(s.users.a1!.power).toBe(expected[tier].power);
   expect(s.sides.A.progress).toBeGreaterThanOrEqual(expected[tier].progress - 1e-6);
   expect(s.sides.A.progress).toBeLessThanOrEqual(expected[tier].progress + 1e-6);
  });
 }
 it('no single gift exceeds 12.5% of the win line', () => {
  for (const t of cfg.tiers) expect(t.progress).toBeLessThanOrEqual(cfg.winProgress * 0.125);
 });
 it('handles a 66x combo of T0', () => {
  const s = clashRoom();
  const fx = gift(s, 'a1', 0, { count: 66 });
```

```
 * 可选：--port=7788 --room=demo-room
 */
import { DEFAULT_CONFIG, type PlatformAdapter } from '@jinli/shared';
import { LocalGrabAdapter } from './adapters/local-grab/adapter';
import { SimAdapter } from './adapters/sim-adapter';
import { startBroadcast } from './broadcast';
import { RoomManager } from './room-manager';
function arg(name: string, fallback: string): string {
  const hit = process.argv.find((a) => a.startsWith(`--${name}=`));
  return hit ? hit.slice(name.length + 3) : fallback;
}
const log = (msg: string) => console.log(`[${new Date().toISOString()}] ${msg}`);
async function main(): Promise<void> {
  const source = arg('source', 'sim');
  const port = Number(arg('port', '7788'));
  const roomId = arg('room', 'demo-room');
  const rooms = new RoomManager({ config: DEFAULT_CONFIG, idleRoomMs: 10 * 60 * 1000 });
  rooms.start();
  let adapter: PlatformAdapter;
  if (source === 'local-grab') {
    log('source=local-grab（仅限 V0 验证轨：可弃置测试号，T0 后停用）');
    adapter = new LocalGrabAdapter({ url: arg('grab-url', `ws://127.0.0.1:1088/ws/${arg('douyin-room', roomId)}`), roomId, log });
  } else {
    adapter = new SimAdapter({ roomId });
  }
  adapter.onEvent((e) => rooms.ingest(e));
  adapter.onStatus((s) => log(`adapter ${adapter.name}: ${s.state}${s.state === 'disconnected' ? ` retry in ${s.retryInMs}ms` : ''}`));
  const server = await startBroadcast(rooms, port, roomId);
  await adapter.connect();
  log(`broadcast ws://127.0.0.1:${server.port}/?room=${roomId}`);
  const statsTimer = setInterval(() => {
    log(`stats events=${rooms.stats.events} frames=${rooms.stats.frames} clients=${server.clients()} rooms=${rooms.roomIds().length}`);
  }, 30_000);
  const shutdown = async () => {
    clearInterval(statsTimer);
    rooms.stop();
    await adapter.close();
    await server.close();
    process.exit(0);
  };
  process.on('SIGINT', () => void shutdown());
  process.on('SIGTERM', () => void shutdown());
}
void main();
// server/src/room-manager.ts
/**
 * RoomManager：core 内核的外壳——房间生命周期、时钟驱动、配置热更新、快照广播。
 * 这里允许读实时钟与定时器（内核不允许）；时钟与定时器均可注入，便于测试与压测。
 */
import { createRoomState, snapshot, step, type CoreInput, type Effect, type RoomState, type Snapshot } from '@jinli/core';
```

```
import type { GameConfig, LiveEvent } from '@jinli/shared';
import { parseGameConfig } from '@jinli/shared/schema';
export interface Frame {
  type: 'frame';
  roomId: string;
  seq: number;
  /** 广播发出时间（服务端时钟）。 */
  sentAt: number;
  snapshot: Snapshot;
  effects: Effect[];
}
export type FrameListener = (frame: Frame) => void;
interface Room {
  id: string;
  state: RoomState;
  config: GameConfig;
  pending: Effect[];
  listeners: Set<FrameListener>;
  seq: number;
  lastActivity: number;
}
export interface Scheduler {
  setInterval(fn: () => void, ms: number): unknown;
  clearInterval(handle: unknown): void;
}
export interface RoomManagerOptions {
  config: GameConfig;
  now?: () => number;
  tickMs?: number;
  broadcastMs?: number;
  /** 无订阅且无事件超过该时长的房间被回收。 */
  idleRoomMs?: number;
  scheduler?: Scheduler;
}
const realScheduler: Scheduler = {
  setInterval: (fn, ms) => setInterval(fn, ms),
  clearInterval: (h) => clearInterval(h as NodeJS.Timeout),
};
export class RoomManager {
  private readonly rooms = new Map<string, Room>();
  private config: GameConfig;
  private pendingConfig: GameConfig | null = null;
  private readonly now: () => number;
  private readonly scheduler: Scheduler;
  private handles: unknown[] = [];
  readonly stats = { events: 0, frames: 0, roomsCreated: 0, roomsClosed: 0 };
  constructor(private readonly opts: RoomManagerOptions) {
    this.config = parseGameConfig(opts.config);
    this.now = opts.now ?? Date.now;
    this.scheduler = opts.scheduler ?? realScheduler;
```

```
}
 start(): void {
  if (this.handles.length > 0) return;
  this.handles.push(this.scheduler.setInterval(() => this.tickAll(), this.opts.tickMs ?? 200));
  this.handles.push(this.scheduler.setInterval(() => this.flushAll(), this.opts.broadcastMs ?? 100));
 }
 stop(): void {
  for (const h of this.handles) this.scheduler.clearInterval(h);
  this.handles = [];
 }
 /** 事件入口：按 roomId 路由，房间不存在则开局。 */
 ingest(event: LiveEvent): void {
  this.stats.events++;
  const room = this.ensureRoom(event.roomId);
  room.lastActivity = this.now();
  this.apply(room, event);
 }
 /** 配置热更新：校验后按策略生效（立即 / 下一局）。返回生效方式。 */
 setConfig(input: unknown): 'immediate' | 'next-match' {
  const cfg = parseGameConfig(input);
  this.config = cfg;
  if (cfg.hotReload === 'immediate') {
    for (const room of this.rooms.values()) room.config = cfg;
    this.pendingConfig = null;
    return 'immediate';
  }
  this.pendingConfig = cfg;
  return 'next-match';
 }
 subscribe(roomId: string, listener: FrameListener): () => void {
  const room = this.ensureRoom(roomId);
  room.listeners.add(listener);
  return () => {
    room.listeners.delete(listener);
    room.lastActivity = this.now();
  };
 }
 getSnapshot(roomId: string): Snapshot | null {
  const room = this.rooms.get(roomId);
  return room ? snapshot(room.state, room.config) : null;
 }
 getRoomConfigVersion(roomId: string): string | null {
  return this.rooms.get(roomId)?.config.version ?? null;
 }
 roomIds(): string[] {
  return [...this.rooms.keys()];
 }
 closeRoom(roomId: string): void {
  if (this.rooms.delete(roomId)) this.stats.roomsClosed++;
 }
```

```
 /** 时钟推进所有房间；同时回收闲置房间。 */
 tickAll(): void {
  const now = this.now();
  for (const room of this.rooms.values()) {
   this.apply(room, { type: 'tick', ts: now });
   const idle = this.opts.idleRoomMs;
   if (idle !== undefined && room.listeners.size === 0 && now - room.lastActivity > idle) this.closeRoom(room.id);
  }
 }
 /** 广播：每房间合并本周期效果 + 最新快照为一帧。 */
 flushAll(): void {
  const sentAt = this.now();
  for (const room of this.rooms.values()) {
   if (room.listeners.size === 0) {
    room.pending = [];
    continue;
   }
   const frame: Frame = {
    type: 'frame',
    roomId: room.id,
    seq: room.seq++,
    sentAt,
    snapshot: snapshot(room.state, room.config),
    effects: room.pending,
   };
   room.pending = [];
   this.stats.frames++;
   for (const l of room.listeners) l(frame);
  }
 }
 private apply(room: Room, input: CoreInput): void {
  // 下一局生效：房间处于结算期即换上新配置，新局从开局起按新数值结算
  if (this.pendingConfig && room.state.phase === 'settle' && room.config !== this.pendingConfig) {
   room.config = this.pendingConfig;
  }
  const { effects } = step(room.state, input, room.config);
  if (effects.length > 0) room.pending.push(...effects);
 }
 private ensureRoom(roomId: string): Room {
  let room = this.rooms.get(roomId);
  if (!room) {
   const now = this.now();
   room = {
    id: roomId,
    state: createRoomState(roomId, now),
    config: this.config,
    pending: [],
    listeners: new Set(),
    seq: 0,
    lastActivity: now,
```

```
    };
    this.rooms.set(roomId, room);
    this.stats.roomsCreated++;
   }
   return room;
  }
}
// server/test/room-manager.test.ts
import { describe, expect, it } from 'vitest';
import { DEFAULT_SIM, generateSimEvents, makeConfig } from '@jinli/shared';
import { RoomManager, type Frame, type Scheduler } from '../src';
function fakeEnv() {
 let now = 0;
 const jobs: { fn: () => void; ms: number; next: number; id: number }[] = [];
 let ids = 0;
 const scheduler: Scheduler = {
  setInterval: (fn, ms) => {
   const id = ids++;
   jobs.push({ fn, ms, next: now + ms, id });
   return id;
  },
  clearInterval: (h) => {
   const i = jobs.findIndex((j) => j.id === h);
   if (i >= 0) jobs.splice(i, 1);
  },
 };
 const advanceTo = (t: number) => {
  for (;;) {
   const job = jobs.filter((j) => j.next <= t).sort((a, b) => a.next - b.next)[0];
   if (!job) break;
   now = job.next;
   job.next += job.ms;
   job.fn();
  }
  now = t;
 };
 return { scheduler, now: () => now, advanceTo };
}
describe('RoomManager', () => {
 it('feeds 5000 random events and broadcasts a consistent frame sequence', () => {
  const env = fakeEnv();
  const rm = new RoomManager({ config: makeConfig(), now: env.now, scheduler: env.scheduler });
  rm.start();
  const frames: Frame[] = [];
  const unsub = rm.subscribe('demo-room', (f) => frames.push(f));
  const events = generateSimEvents({ ...DEFAULT_SIM, startTs: 0, durationMs: 25 * 60 * 1000, eventsPerSec: 3.5 }).slice(0, 5000);
  expect(events).toHaveLength(5000);
  for (const ev of events) {
   env.advanceTo(ev.ts);
   rm.ingest(ev);
```

```
  }
  env.advanceTo(events.at(-1)!.ts + 1000);
  expect(rm.stats.events).toBe(5000);
  expect(frames.length).toBeGreaterThan(100);
  for (let i = 1; i < frames.length; i++) {
   expect(frames[i]!.seq).toBe(frames[i - 1]!.seq + 1);
   expect(frames[i]!.snapshot.matchNo).toBeGreaterThanOrEqual(frames[i - 1]!.snapshot.matchNo);
  }
  const effectCount = frames.reduce((n, f) => n + f.effects.length, 0);
  expect(effectCount).toBeGreaterThan(5000);
  unsub();
  rm.stop();
 });
 it('applies hot-reloaded config on the next match by default', () => {
  const env = fakeEnv();
  const rm = new RoomManager({ config: makeConfig(), now: env.now, scheduler: env.scheduler });
  rm.start();
  rm.subscribe('r', () => {});
  expect(rm.setConfig(makeConfig({ version: 'v2' }))).toBe('next-match');
  expect(rm.getRoomConfigVersion('r')).toBe('5.9-v1+v0.7');
  env.advanceTo(30 * 60 * 1000); // 空房间：蓄势→攻防→鸿运→终局潮汐→结算→下一局
  expect(rm.getSnapshot('r')!.matchNo).toBeGreaterThan(1);
  expect(rm.getRoomConfigVersion('r')).toBe('v2');
 });
 it('applies immediately when the config says so, and rejects invalid configs', () => {
  const env = fakeEnv();
  const rm = new RoomManager({ config: makeConfig(), now: env.now, scheduler: env.scheduler });
  rm.subscribe('r', () => {});
  expect(rm.setConfig(makeConfig({ version: 'hot', hotReload: 'immediate' }))).toBe('immediate');
  expect(rm.getRoomConfigVersion('r')).toBe('hot');
  const bad = makeConfig();
  bad.tiers[7].progress = 99999;
  expect(() => rm.setConfig(bad)).toThrow();
 });
 it('recycles idle rooms with no subscribers', () => {
  const env = fakeEnv();
  const rm = new RoomManager({ config: makeConfig(), now: env.now, scheduler: env.scheduler, idleRoomMs: 5000 });
  rm.start();
  const unsub = rm.subscribe('r', () => {});
  env.advanceTo(10_000);
  expect(rm.roomIds()).toEqual(['r']);
  unsub();
  env.advanceTo(20_000);
  expect(rm.roomIds()).toEqual([]);
  rm.stop();
 });
});
// client/assets/scripts/logic/layout.ts
/**
 * 布局档案（Layout Profile）：画布、安全区、各区域锚点全部来自 JSON 档案，
```

```
 * 切换平台档案不改代码。输出为以画布左上角为原点的矩形（y 向下）。
 */
export type Anchor = 'top' | 'bottom' | 'center' | 'top-left' | 'top-right';
export interface RegionSpec {
  anchor: Anchor;
  offsetY?: number;
  width?: number;
  height?: number;
  heightRatio?: number;
}
export interface LayoutProfile {
  name: string;
  canvas: { width: number; height: number };
  safeArea: { top: number; bottom: number; left: number; right: number };
  regions: Record<'topBar' | 'rank' | 'battlefield' | 'feed' | 'ruleSticker' | 'watermark', RegionSpec>;
}
export interface Rect {
  x: number;
  y: number;
  w: number;
  h: number;
}
export type ResolvedLayout = Record<keyof LayoutProfile['regions'], Rect>;
const REGION_KEYS: (keyof LayoutProfile['regions'])[] = ['topBar', 'rank', 'battlefield', 'feed', 'ruleSticker', 'watermark'];
export function parseProfile(input: unknown): LayoutProfile {
  const p = input as Partial<LayoutProfile> | null;
  if (!p || typeof p !== 'object') throw new Error('profile: not an object');
  if (typeof p.name !== 'string') throw new Error('profile: name missing');
  const c = p.canvas;
  if (!c || !(c.width > 0) || !(c.height > 0)) throw new Error('profile: invalid canvas');
  const s = p.safeArea;
  if (!s || [s.top, s.bottom, s.left, s.right].some((v) => typeof v !== 'number' || v < 0)) throw new Error('profile: invalid safeArea');
  if (s.top + s.bottom >= c.height || s.left + s.right >= c.width) throw new Error('profile: safeArea larger than canvas');
  for (const k of REGION_KEYS) if (!p.regions?.[k]) throw new Error(`profile: region ${k} missing`);
  return p as LayoutProfile;
}
export function resolveLayout(profile: LayoutProfile): ResolvedLayout {
  const { canvas, safeArea: s, regions } = profile;
  const innerW = canvas.width - s.left - s.right;
  const innerH = canvas.height - s.top - s.bottom;
  const out = {} as ResolvedLayout;
  for (const key of REGION_KEYS) {
    const r = regions[key];
    const h = r.heightRatio !== undefined ? innerH * r.heightRatio : r.height ?? 24;
    const w = r.width ?? innerW;
    const off = r.offsetY ?? 0;
    let x = s.left;
    let y = s.top;
    switch (r.anchor) {
      case 'top':
```

```
      y = s.top + off;
      break;
    case 'top-left':
      y = s.top + off;
      break;
    case 'top-right':
      x = canvas.width - s.right - w;
      y = s.top + off;
      break;
    case 'bottom':
      y = canvas.height - s.bottom - h - off;
      break;
    case 'center':
      y = s.top + (innerH - h) / 2 + off;
      break;
   }
   if (r.anchor === 'top' || r.anchor === 'bottom' || r.anchor === 'center') x = s.left + (innerW - w) / 2;
   out[key] = { x, y, w, h };
  }
  return out;
}
// client/assets/scripts/logic/sources.ts
/**
 * 数据源：demo（本地内核 + 模拟剧本，无 server 完整演示）与 net（WS 消费 server 帧）。
 */
import { DEFAULT_CONFIG, DEFAULT_SIM, generateSimEvents, type GameConfig, type LiveEvent } from '../generated/shared';
import { createRoomState, snapshot, step, type Effect, type RoomState, type Snapshot } from '../generated/core';
export interface FrameData {
  snapshot: Snapshot;
  effects: Effect[];
}
export interface FrameSource {
  /** 推进到 nowMs（demo 为虚拟时钟），并返回自上次以来的帧（可能为空）。 */
  poll(nowMs: number): FrameData | null;
}
export interface DemoOptions {
  roomId?: string;
  seed?: number;
  /** 虚拟时间加速倍率（演示一局只需 ~14 分钟，可加速）。 */
  speed?: number;
  config?: GameConfig;
}
/** 本地内核驱动的演示源：无 server 也能完整跑一局。 */
export class DemoSource implements FrameSource {
  private readonly config: GameConfig;
  private readonly state: RoomState;
  private events: LiveEvent[];
  private cursor = 0;
  private loops = 0;
  private readonly seed: number;
```

```
  private readonly roomId: string;
  private virtual = 0;
  private lastRealMs: number | null = null;
  private readonly speed: number;
  private lastTick = 0;
  constructor(opts: DemoOptions = {}) {
   this.config = opts.config ?? DEFAULT_CONFIG;
   this.speed = opts.speed ?? 1;
   this.roomId = opts.roomId ?? 'demo-room';
   this.seed = opts.seed ?? DEFAULT_SIM.seed;
   this.state = createRoomState(this.roomId, 0);
   this.events = this.script(0);
  }
  poll(nowMs: number): FrameData | null {
   const dt = this.lastRealMs === null ? 0 : nowMs - this.lastRealMs;
   this.lastRealMs = nowMs;
   this.virtual += dt * this.speed;
   const effects: Effect[] = [];
   while (this.cursor < this.events.length && (this.events[this.cursor]?.ts ?? Infinity) <= this.virtual) {
    effects.push(...step(this.state, this.events[this.cursor++]!, this.config).effects);
   }
   while (this.lastTick + 200 <= this.virtual) {
    this.lastTick += 200;
    effects.push(...step(this.state, { type: 'tick', ts: this.lastTick }, this.config).effects);
   }
   // 剧本播完后接续：从当前虚拟时间起生成下一段新剧本（换种子），演示不断且不重放旧事件
   if (this.cursor >= this.events.length) {
    this.loops++;
    this.events = this.script(this.virtual);
    this.cursor = 0;
   }
   return { snapshot: snapshot(this.state, this.config), effects };
  }
  private script(startTs: number): LiveEvent[] {
   return generateSimEvents({ ...DEFAULT_SIM, seed: this.seed + this.loops, roomId: this.roomId, startTs, durationMs: 14 * 60 * 1000 });
  }
  /** 当前内核状态（测试/截图用）。 */
  get matchNo(): number {
   return this.state.matchNo;
  }
}
/** 网络源：解析 server 广播的 Frame JSON。 */
export class NetSource implements FrameSource {
  private queue: FrameData[] = [];
  private socket: { close(): void } | null = null;
  constructor(private readonly url: string, private readonly makeSocket: (url: string) => WebSocketLike = (u) => new WebSocket(u)) {}
  connect(onStatus?: (s: 'open' | 'closed') => void): void {
   const ws = this.makeSocket(this.url);
   this.socket = ws;
   ws.onopen = () => onStatus?.('open');
```

```
    ws.onclose = () => onStatus?.('closed');
    ws.onmessage = (m) => this.ingest(typeof m.data === 'string' ? m.data : '');
  }
  ingest(text: string): void {
    try {
      const msg = JSON.parse(text) as { type?: string; snapshot?: Snapshot; effects?: Effect[] };
      if (msg.type === 'frame' && msg.snapshot) this.queue.push({ snapshot: msg.snapshot, effects: msg.effects ?? [] });
    } catch {
      /* 忽略非法帧 */
    }
  }
  poll(): FrameData | null {
    if (this.queue.length === 0) return null;
    const frames = this.queue;
    this.queue = [];
    return { snapshot: frames[frames.length - 1]!.snapshot, effects: frames.flatMap((f) => f.effects) };
  }
  close(): void {
    this.socket?.close();
  }
}
export interface WebSocketLike {
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  onopen: ((ev: any) => void) | null;
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  onclose: ((ev: any) => void) | null;
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  onmessage: ((m: any) => void) | null;
  close(): void;
}
// client/assets/scripts/logic/view-model.ts
/**
 * 视图模型：把 core 的快照与效果转成渲染所需数据与"反馈队列"。
 * 纯逻辑、无 cc 依赖，可在 Node 下单测。外显文案均来自配置标签或固定占位，且过词表。
 */
import { findBanned, maskText } from '../generated/shared';
import type { Effect, Phase, Snapshot } from '../generated/core';
export interface FeedItem {
  id: number;
  at: number;
  text: string;
  tone: 'chat' | 'like' | 'gift' | 'rare' | 'system';
  /** 展示截止时间（毫秒，与 snapshot.now 同一时钟）。 */
  until: number;
}
export interface Banner {
  text: string;
  until: number;
}
export const PHASE_LABEL: Record<Phase, string> = {
```

```
  charge: '蓄势',
  clash: '攻防',
  fortune: '鸿运时刻',
  overtime: '终局潮汐',
  settle: '结算',
};
const FEED_MAX = 6;
const FEED_TTL_MS = 6000;
const BANNER_TTL_MS = 3500;
/** 外显文案出口：统一过词表，命中即打码。 */
export function safeText(text: string): string {
  return findBanned(text).length > 0 ? maskText(text) : text;
}
export function effectToFeed(e: Effect, sideNames: { A: string; B: string }): { text: string; tone: FeedItem['tone'] } | null {
  switch (e.kind) {
    case 'comment':
      return { text: `${e.nick}：${e.text}`, tone: 'chat' };
    case 'like':
      return { text: `${e.nick} 点亮了 ${e.count} 次`, tone: 'like' };
    case 'join':
      return { text: `${e.nick} 加入了${sideNames[e.side]}`, tone: 'system' };
    case 'gift': {
      const skin = e.skins.length > 0 ? `・${e.skins.join('')}` : '';
      const gain = e.progressPct > 0 ? `，进度 +${e.progressPct}%` : '';
      const carry = e.carried ? '（计入下一局）' : '';
      return { text: `${e.nick} ${e.label}${e.count > 1 ? ` ×${e.count}` : ''}${skin}${gain}${carry}`, tone: e.rare ? 'rare' : 'gift' };
    }
    case 'surge':
      return { text: `${sideNames[e.side]} 全民锦鲤潮！`, tone: 'system' };
    case 'counter':
      return { text: '巧！克制命中', tone: 'system' };
    case 'dragon-full':
      return { text: `${e.nick} 的神龙名额已满，改为全队强化`, tone: 'system' };
    default:
      return null;
  }
}
export class ViewModel {
  feed: FeedItem[] = [];
  banner: Banner | null = null;
  snapshot: Snapshot | null = null;
  private nextId = 1;
  constructor(private readonly sideNames: { A: string; B: string }) {}
  /** 应用一帧：更新快照、追加反馈、触发横幅。 */
  apply(snapshot: Snapshot, effects: Effect[]): void {
    this.snapshot = snapshot;
    const now = snapshot.now;
    for (const e of effects) {
      const item = effectToFeed(e, this.sideNames);
      if (item) this.feed.push({ id: this.nextId++, at: e.at, text: safeText(item.text), tone: item.tone, until: now + FEED_TTL_MS });
```

```
    const banner = this.bannerFor(e);
    if (banner) this.banner = { text: safeText(banner), until: now + BANNER_TTL_MS };
   }
   this.feed = this.feed.filter((f) => f.until > now).slice(-FEED_MAX);
   if (this.banner && this.banner.until <= now) this.banner = null;
  }
  private bannerFor(e: Effect): string | null {
   switch (e.kind) {
    case 'phase':
     return e.phase === 'settle' ? null : `${PHASE_LABEL[e.phase]}`;
    case 'leap':
     return `${e.nick} 鲤跃龙门！`;
    case 'settle': {
     const w = e.result.winner;
     return w === 'draw' ? '平分秋色' : `${this.sideNames[w]} 获胜`;
    }
    case 'comeback':
     return e.side ? `${this.sideNames[e.side]} 逆流而上` : null;
    default:
     return null;
   }
  }
  /** 距阶段结束的剩余秒数（终局潮汐为 null）。 */
  countdownSec(): number | null {
   const s = this.snapshot;
   if (!s || s.phaseEndsAt === null) return null;
   return Math.max(0, Math.ceil((s.phaseEndsAt - s.now) / 1000));
  }
}
// client/assets/scripts/view/JinliMain.ts
/**
 * Cocos 视图层（极薄）：运行时用 Graphics/Label 构建全部占位 UI，逻辑全部来自 logic 层。
 * 使用：Canvas 节点挂载本组件即可，无需手写场景内容。画面统一带 DEV 水印。
 */
import { _decorator, Color, Component, Graphics, JsonAsset, Label, Node, resources, UITransform, Vec3 } from 'cc';
import type { Snapshot } from '../generated/core';
import { parseProfile, resolveLayout, type LayoutProfile, type Rect, type ResolvedLayout } from '../logic/layout';
import { DemoSource, NetSource, type FrameSource } from '../logic/sources';
import { PHASE_LABEL, ViewModel } from '../logic/view-model';
const { ccclass, property } = _decorator;
const COLORS = {
 A: new Color(231, 76, 60, 255),
 B: new Color(52, 152, 219, 255),
 panel: new Color(0, 0, 0, 120),
 text: new Color(255, 245, 220, 255),
 gold: new Color(255, 205, 80, 255),
};
@ccclass('JinliMain')
export class JinliMain extends Component {
 @property profileName = 'douyin-375x812';
```

```
@property mode = 'demo';
@property serverUrl = 'ws://127.0.0.1:7788/?room=demo-room';
@property demoSpeed = 1;
private layout!: ResolvedLayout;
private profile!: LayoutProfile;
private source!: FrameSource;
private vm = new ViewModel({ A: '锦鲤', B: '神龟' });
private g!: Graphics;
private labels = new Map<string, Label>();
private elapsedMs = 0;
start(): void {
 resources.load(`profiles/${this.profileName}`, JsonAsset, (err, asset) => {
  if (err || !asset) {
   console.error('layout profile load failed', err);
   return;
  }
  this.profile = parseProfile(asset.json);
  this.layout = resolveLayout(this.profile);
  this.build();
 });
 if (this.mode === 'net') {
  const net = new NetSource(this.serverUrl);
  net.connect();
  this.source = net;
 } else {
  this.source = new DemoSource({ speed: this.demoSpeed });
 }
}
update(dt: number): void {
 if (!this.layout) return;
 this.elapsedMs += dt * 1000;
 const frame = this.source.poll(this.elapsedMs);
 if (frame) this.vm.apply(frame.snapshot, frame.effects);
 this.draw();
}
private build(): void {
 const { canvas } = this.profile;
 const ui = this.node.getComponent(UITransform) ?? this.node.addComponent(UITransform);
 ui.setContentSize(canvas.width, canvas.height);
 const gNode = this.child('graphics');
 this.g = gNode.addComponent(Graphics);
 for (const key of ['phase', 'countdown', 'sideA', 'sideB', 'banner', 'rank', 'feed', 'rule', 'dev']) {
  this.labels.set(key, this.label(key));
 }
}
private child(name: string): Node {
 const n = new Node(name);
 n.layer = this.node.layer;
 this.node.addChild(n);
 return n;
```

```
}
 private label(name: string): Label {
  const node = this.child(name);
  node.addComponent(UITransform);
  const l = node.addComponent(Label);
  l.fontSize = 14;
  l.lineHeight = 18;
  l.color = COLORS.text;
  return l;
 }
 /** 档案坐标（左上原点，y 向下）→ 节点坐标（画布中心原点，y 向上）。 */
 private place(key: string, r: Rect, align: 'left' | 'center' = 'left'): void {
  const l = this.labels.get(key);
  if (!l) return;
  const { canvas } = this.profile;
  const ui = l.node.getComponent(UITransform)!;
  ui.setContentSize(r.w, r.h);
  ui.setAnchorPoint(0.5, 0.5);
  l.horizontalAlign = align === 'center' ? Label.HorizontalAlign.CENTER : Label.HorizontalAlign.LEFT;
  l.verticalAlign = Label.VerticalAlign.CENTER;
  l.overflow = Label.Overflow.SHRINK;
  l.node.setPosition(new Vec3(r.x + r.w / 2 - canvas.width / 2, canvas.height / 2 - (r.y + r.h / 2), 0));
 }
 private rect(g: Graphics, r: Rect, color: Color): void {
  const { canvas } = this.profile;
  g.fillColor = color;
  g.rect(r.x - canvas.width / 2, canvas.height / 2 - r.y - r.h, r.w, r.h);
  g.fill();
 }
 private draw(): void {
  const snap = this.vm.snapshot;
  if (!snap) return;
  const L = this.layout;
  const g = this.g;
  g.clear();
  this.rect(g, L.topBar, COLORS.panel);
  // 进度条：左锦鲤（红）/右神龟（蓝）
  const barH = 14;
  const barY = L.topBar.y + L.topBar.h - barH - 6;
  const barW = L.topBar.w;
  const half = barW / 2;
  this.rect(g, { x: L.topBar.x, y: barY, w: barW, h: barH }, new Color(40, 40, 40, 200));
  this.rect(g, { x: L.topBar.x + half - half * snap.sides.A.progress, y: barY, w: half * snap.sides.A.progress, h: barH }, COLORS.A);
  this.rect(g, { x: L.topBar.x + half, y: barY, w: half * snap.sides.B.progress, h: barH }, COLORS.B);
  // 集体充能条
  for (const [side, color] of [['A', COLORS.A], ['B', COLORS.B]] as const) {
   const w = (half - 8) * snap.sides[side].charge;
   const x = side === 'A' ? L.topBar.x : L.topBar.x + half + 8;
   this.rect(g, { x, y: barY + barH + 2, w, h: 3 }, color);
  }
```

```
  // 战场：双方小鱼占位（按人数与鱼苗数）
  this.drawFish(g, snap, L.battlefield);
  this.rect(g, L.feed, COLORS.panel);
  this.rect(g, L.ruleSticker, new Color(0, 0, 0, 160));
  const sec = this.vm.countdownSec();
  this.setText('phase', `${PHASE_LABEL[snap.phase]} · 第${snap.matchNo}场`, L.topBar, 'left', { x: 0, y: 4, h: 20 });
  this.setText('countdown', sec === null ? '' : `${Math.floor(sec / 60)}:${String(sec % 60).padStart(2, '0')}`, L.topBar, 'center', { x: 0, y: 4, h: 20 });
  const side = (s: 'A' | 'B') => `${snap.sides[s].name} ${Math.round(snap.sides[s].progress * 100)}%  ${snap.sides[s].members}人
${snap.sides[s].comeback ? '↑' : ''}`;
  this.setText('sideA', side('A'), L.topBar, 'left', { x: 0, y: 24, h: 18 });
  this.setText('sideB', side('B'), L.topBar, 'left', { x: L.topBar.w / 2, y: 24, h: 18 });
  this.setText('rank', snap.top.map((r, i) => `${i + 1} ${r.nick} ${r.power}`).join('\n'), L.rank, 'left');
  this.setText('feed', this.vm.feed.map((f) => f.text).join('\n'), L.feed, 'left');
  this.setText('banner', this.vm.banner?.text ?? '', L.battlefield, 'center', { x: 0, y: 0, h: 40 }, 28);
  this.setText('rule', '弹幕"锦鲤"或"神龟"加入 · 点赞助力 · 礼物召唤', L.ruleSticker, 'center');
  this.setText('dev', 'DEV', L.watermark, 'left', { x: 0, y: 0, w: 40, h: 18 });
 }
 private drawFish(g: Graphics, snap: Snapshot, field: Rect): void {
  const { canvas } = this.profile;
  for (const [i, side] of (['A', 'B'] as const).entries()) {
   const n = Math.min(40, snap.sides[side].members + snap.sides[side].fry * 3);
   const laneY = field.y + field.h * (0.3 + 0.4 * i);
   g.fillColor = side === 'A' ? COLORS.A : COLORS.B;
   for (let k = 0; k < n; k++) {
    const t = (k + 1) / (n + 1);
    const x = field.x + 12 + t * (field.w - 24) * (0.2 + 0.8 * snap.sides[side].progress);
    g.circle(x - canvas.width / 2, canvas.height / 2 - laneY + ((k % 5) - 2) * 6, 5);
   }
   g.fill();
  }
 }
 private setText(key: string, text: string, region: Rect, align: 'left' | 'center', box: { x?: number; y?: number; w?: number; h?: number } = {}, size?:
number): void {
  const l = this.labels.get(key);
  if (!l) return;
  l.string = text;
  if (size) l.fontSize = size;
  this.place(key, { x: region.x + (box.x ?? 0), y: region.y + (box.y ?? 0), w: box.w ?? region.w - (box.x ?? 0), h: box.h ?? region.h }, align);
 }
}
// client/test/client.test.ts
import { describe, expect, it } from 'vitest';
import { findBanned } from '../assets/scripts/generated/shared';
import { parseProfile, resolveLayout } from '../assets/scripts/logic/layout';
import { DemoSource, NetSource } from '../assets/scripts/logic/sources';
import { PHASE_LABEL, ViewModel, safeText } from '../assets/scripts/logic/view-model';
import douyin from '../assets/resources/profiles/douyin-375x812.json';
import wide from '../assets/resources/profiles/test-wide.json';
const names = { A: '锦鲤', B: '神龟' };
describe('layout profiles', () => {
 it('resolves regions inside the canvas for every shipped profile', () => {
  for (const raw of [douyin, wide]) {
```

```
    const p = parseProfile(raw);
    const l = resolveLayout(p);
    for (const [k, r] of Object.entries(l)) {
      expect(r.x, `${p.name}.${k}`).toBeGreaterThanOrEqual(0);
      expect(r.y, `${p.name}.${k}`).toBeGreaterThanOrEqual(0);
      expect(r.x + r.w, `${p.name}.${k}`).toBeLessThanOrEqual(p.canvas.width + 1e-6);
      expect(r.y + r.h, `${p.name}.${k}`).toBeLessThanOrEqual(p.canvas.height + 1e-6);
    }
  }
 });
 it('changing the safe area moves the layout with zero code changes', () => {
  const base = resolveLayout(parseProfile(douyin));
  const moved = resolveLayout(parseProfile({ ...douyin, safeArea: { ...douyin.safeArea, top: douyin.safeArea.top + 40 } }));
  expect(moved.topBar.y - base.topBar.y).toBe(40);
 });
 it('switching profile changes anchors (rank panel moves to the other corner)', () => {
  const a = resolveLayout(parseProfile(douyin));
  const b = resolveLayout(parseProfile(wide));
  expect(a.rank.x).toBeLessThan(a.topBar.x + 20);
  expect(b.rank.x).toBeGreaterThan(b.topBar.x + b.topBar.w / 2);
 });
 it('rejects broken profiles', () => {
  expect(() => parseProfile({ name: 'x' })).toThrow();
  expect(() => parseProfile({ ...douyin, safeArea: { top: 900, bottom: 0, left: 0, right: 0 } })).toThrow(/larger/);
 });
});
describe('demo mode', () => {
 it('plays a full match (charge → clash → fortune → settle) without a server', () => {
  const src = new DemoSource({ seed: 11 });
  const vm = new ViewModel(names);
  const phases = new Set<string>();
  let settled = false;
  for (let t = 0; t <= 25 * 60 * 1000; t += 100) {
   const f = src.poll(t);
   if (!f) continue;
   vm.apply(f.snapshot, f.effects);
   phases.add(f.snapshot.phase);
   if (f.snapshot.result) settled = true;
  }
  expect([...phases]).toEqual(expect.arrayContaining(['charge', 'clash', 'fortune']));
  expect(settled).toBe(true);
  expect(src.matchNo).toBeGreaterThanOrEqual(1);
 });
 it('reflects state changes in the very next frame (≤1s budget)', () => {
  const src = new DemoSource({ seed: 3 });
  let prev = src.poll(0)!.snapshot;
  let changedWithin = Infinity;
  for (let t = 100; t <= 120_000; t += 100) {
   const f = src.poll(t)!;
   if (f.effects.some((e) => e.kind === 'gift')) {
```

```
      changedWithin = 100; // 效果与快照同帧产出
      expect(f.snapshot.now).toBeGreaterThan(prev.now);
      break;
    }
    prev = f.snapshot;
  }
  expect(changedWithin).toBeLessThanOrEqual(1000);
 });
});
describe('demo loop', () => {
 it('continues with a fresh script after 14 minutes instead of replaying old events in one burst', () => {
  const src = new DemoSource({ seed: 9 });
  let maxEffects = 0;
  for (let t = 0; t <= 30 * 60 * 1000; t += 100) maxEffects = Math.max(maxEffects, src.poll(t)!.effects.length);
  expect(maxEffects).toBeLessThan(200);
  expect(src.matchNo).toBeGreaterThanOrEqual(2);
 });
});
describe('feedback', () => {
 it('turns a gift effect into feed text with the masked nick, in the same frame', () => {
  const vm = new ViewModel(names);
  const snap = new DemoSource().poll(0)!.snapshot;
  vm.apply(snap, [{ kind: 'gift', at: snap.now, side: 'A', nick: '小鱼**', tier: 4, effectTier: 4, count: 1, label: '召唤锦鲤', progressPct: 12.5, skins: ['星耀鲤'], rare: true, carried: false }]);
  expect(vm.feed.at(-1)).toMatchObject({ tone: 'rare', text: '小鱼** 召唤锦鲤・星耀鲤，进度 +12.5%' });
 });
 it('expires feed items and caps the list', () => {
  const vm = new ViewModel(names);
  const snap = new DemoSource().poll(0)!.snapshot;
  const comment = (i: number) => ({ kind: 'comment' as const, at: 0, side: 'A' as const, nick: 'n', text: `c${i}` });
  vm.apply(snap, Array.from({ length: 20 }, (_, i) => comment(i)));
  expect(vm.feed).toHaveLength(6);
  vm.apply({ ...snap, now: snap.now + 10_000 }, []);
  expect(vm.feed).toHaveLength(0);
 });
 it('never renders banned words, even from raw user text', () => {
  expect(findBanned(safeText('来抽奖吧'))).toEqual([]);
  for (const label of Object.values(PHASE_LABEL)) expect(findBanned(label)).toEqual([]);
 });
 it('never exposes money units in the feed', () => {
  const vm = new ViewModel(names);
  const src = new DemoSource({ seed: 5, speed: 20 });
  for (let t = 0; t < 60_000; t += 100) {
   const f = src.poll(t)!;
   vm.apply(f.snapshot, f.effects);
   for (const item of vm.feed) expect(item.text).not.toMatch(/[元￥¥]|抖币|\d+\s*币/);
  }
 });
});
describe('net source', () => {
 it('queues frames, merges effects, and ignores garbage', () => {
```

```
  const net = new NetSource('ws://x', () => ({ onopen: null, onclose: null, onmessage: null, close() {} }));
  const snap = new DemoSource().poll(0)!.snapshot;
  net.ingest('{bad');
  net.ingest(JSON.stringify({ type: 'hello' }));
  net.ingest(JSON.stringify({ type: 'frame', snapshot: snap, effects: [{ kind: 'follow', at: 0, nick: 'a' }] }));
  net.ingest(JSON.stringify({ type: 'frame', snapshot: { ...snap, now: 5 }, effects: [{ kind: 'follow', at: 0, nick: 'b' }] }));
  const f = net.poll()!;
  expect(f.effects).toHaveLength(2);
  expect(f.snapshot.now).toBe(5);
  expect(net.poll()).toBeNull();
 });
});
// client/test/stubs/cc.d.ts
/**
 * Cocos Creator 3.8 `cc` 模块的最小类型桩——仅用于在 CI 里对视图层做类型检查（API 形状按 3.8 文档）。
 * 不是真实运行时；视图层真正的编译与运行仍需在编辑器验证（任务 5.1）。
 */
declare module 'cc' {
 export class Color {
  constructor(r?: number, g?: number, b?: number, a?: number);
 }
 export class Vec3 {
  constructor(x?: number, y?: number, z?: number);
 }
 export class Node {
  constructor(name?: string);
  layer: number;
  addChild(child: Node): void;
  addComponent<T extends Component>(type: new () => T): T;
  getComponent<T extends Component>(type: new () => T): T | null;
  setPosition(p: Vec3): void;
 }
 export class Component {
  node: Node;
 }
 export class UITransform extends Component {
  setContentSize(w: number, h: number): void;
  setAnchorPoint(x: number, y: number): void;
 }
 export class Graphics extends Component {
  fillColor: Color;
  clear(): void;
  rect(x: number, y: number, w: number, h: number): void;
  circle(x: number, y: number, r: number): void;
  fill(): void;
 }
 export class Label extends Component {
  static HorizontalAlign: { LEFT: number; CENTER: number; RIGHT: number };
  static VerticalAlign: { TOP: number; CENTER: number; BOTTOM: number };
  static Overflow: { NONE: number; CLAMP: number; SHRINK: number; RESIZE_HEIGHT: number };
```

```
   string: string;
   fontSize: number;
   lineHeight: number;
   color: Color;
   horizontalAlign: number;
   verticalAlign: number;
   overflow: number;
  }
  export class JsonAsset {
   json: unknown;
  }
  export const resources: {
   load<T>(path: string, type: new () => T, cb: (err: Error | null, asset: T | null) => void): void;
  };
  export const _decorator: {
   ccclass(name?: string): ClassDecorator;
   property: PropertyDecorator & ((opts?: unknown) => PropertyDecorator);
  };
}
// tools/ci.ts
/**
 * 本地等价 CI：lint → typecheck → test → scan → sync-check，结果写入 artifacts/verification/。
 * 任一门禁失败则整体失败（但所有门禁都会跑完，便于一次看全）。
 */
import { spawnSync } from 'node:child_process';
import { mkdirSync, writeFileSync } from 'node:fs';
import { resolve } from 'node:path';
const ROOT = resolve(import.meta.dirname, '..');
const OUT = resolve(ROOT, 'artifacts/verification');
mkdirSync(OUT, { recursive: true });
interface Gate {
 name: string;
 cmd: string[];
}
const gates: Gate[] = [
 { name: 'lint', cmd: ['npx', 'eslint', '.'] },
 { name: 'typecheck', cmd: ['npx', 'tsc', '-p', 'tsconfig.json', '--noEmit'] },
 { name: 'typecheck-view', cmd: ['npx', 'tsc', '-p', 'tsconfig.view.json'] },
 { name: 'test', cmd: ['npx', 'vitest', 'run', '--reporter=default', '--reporter=json', `--outputFile=${OUT}/test-report.json`] },
 { name: 'scan', cmd: ['npx', 'tsx', 'tools/scan-wordlist.ts', `--json=${OUT}/scan-result.json`] },
 { name: 'sync-check', cmd: ['npx', 'tsx', 'tools/sync-client.ts', '--check', `--json=${OUT}/sync-check.json`] },
];
const results: { name: string; ok: boolean; ms: number }[] = [];
for (const g of gates) {
 const t0 = Date.now();
 console.log(`\n—— ${g.name} ——`);
 const r = spawnSync(g.cmd[0]!, g.cmd.slice(1), { cwd: ROOT, stdio: 'inherit' });
 results.push({ name: g.name, ok: r.status === 0, ms: Date.now() - t0 });
}
const ok = results.every((r) => r.ok);
```

```
writeFileSync(
 resolve(OUT, 'manifest.json'),
 JSON.stringify({ ok, finishedAt: new Date().toISOString(), node: process.version, gates: results }, null, 2),
);
console.log(`\nCI ${ok ? 'PASSED' : 'FAILED'}: ${results.map((r) => `${r.name}=${r.ok ? 'ok' : 'FAIL'}`).join(' ')}`);
process.exit(ok ? 0 : 1);
// tools/loadtest.ts
/**
 * 管道压测（任务 3.6）：单房间 1000 QPS 混合流量 × 30s，经 local-grab normalizer → RoomManager → 广播。
 * 统计：投递丢失（去重除外）与"入站→广播"延迟 P995。
 * 进程内压测，不含真实网络；全局 4200 QPS 与网络压测后置至 compliance-testkit。
 */
import { mkdirSync, writeFileSync } from 'node:fs';
import { resolve } from 'node:path';
import { DEFAULT_CONFIG } from '../packages/shared/src';
import { LocalGrabNormalizer } from '../server/src/adapters/local-grab/normalizer';
import { DOUYIN_CATALOG } from '../server/src/gift-catalog';
import { RoomManager } from '../server/src/room-manager';
const QPS = Number(process.argv.find((a) => a.startsWith('--qps='))?.slice(6) ?? 1000);
const SECONDS = Number(process.argv.find((a) => a.startsWith('--seconds='))?.slice(10) ?? 30);
const OUT = process.argv.find((a) => a.startsWith('--json='))?.slice(7);
async function main(): Promise<void> {
 const rooms = new RoomManager({ config: DEFAULT_CONFIG });
 rooms.start();
 const normalizer = new LocalGrabNormalizer({ roomId: 'load', catalog: DOUYIN_CATALOG, now: Date.now });
 let sent = 0;
 let duplicates = 0;
 let delivered = 0;
 // 每条事件的"入站→广播"延迟：入站时记录时间戳，下一次广播帧发出时结算（帧携带了这些事件的效果与最新快照）
 let arrivals: number[] = [];
 const latencies: number[] = [];
 rooms.subscribe('load', () => {
  const now = performance.now();
  for (const a of arrivals) latencies.push(now - a);
  arrivals = [];
 });
 const methods = ['WebcastChatMessage', 'WebcastLikeMessage', 'WebcastGiftMessage', 'WebcastMemberMessage'] as const;
 const t0 = performance.now();
 let n = 0;
 const batchMs = 10;
 const perBatch = (QPS * batchMs) / 1000;
 await new Promise<void>((resolveRun) => {
  const timer = setInterval(() => {
   const elapsed = performance.now() - t0;
   if (elapsed >= SECONDS * 1000) {
    clearInterval(timer);
    resolveRun();
    return;
   }
   for (let i = 0; i < perBatch; i++) {
```

```
      n++;
      const method = methods[n % 4]!;
      const dup = n % 50 === 0; // 2% 重复推送
      const msgId = dup ? `m${n - 1}` : `m${n}`;
      const payload: Record<string, unknown> = { common: { msgId }, user: { id: n % 500, nickName: `观众${n % 500}` } };
      if (method === 'WebcastChatMessage') payload.content = n % 3 ? '冲' : '锦鲤';
      if (method === 'WebcastLikeMessage') payload.count = 3;
      if (method === 'WebcastGiftMessage') payload.gift = { id: n % 7, diamondCount: [1, 19, 52, 99][n % 4] };
      const arrival = performance.now();
      const events = normalizer.normalize({ method, payload });
      sent++;
      if (dup) duplicates++;
      for (const ev of events) {
        const before = rooms.stats.events;
        rooms.ingest(ev);
        delivered += rooms.stats.events - before;
        arrivals.push(arrival);
      }
    }
  }, batchMs);
 });
 await new Promise((r) => setTimeout(r, 300));
 rooms.stop();
 const expected = sent - duplicates;
 const lost = expected - delivered;
 const framed = latencies.length;
 const sorted = [...latencies].sort((a, b) => a - b);
 const p995 = sorted.length ? sorted[Math.min(sorted.length - 1, Math.floor(sorted.length * 0.995))]! : 0;
 const report = {
  qps: QPS,
  seconds: SECONDS,
  sent,
  duplicatesInjected: duplicates,
  delivered,
  lost,
  measured: framed,
  p995Ms: Math.round(p995 * 10) / 10,
  ok: lost === 0 && framed >= delivered * 0.95 && p995 < 1000,
 };
 console.log(JSON.stringify(report, null, 2));
 if (OUT) {
  mkdirSync(resolve(OUT, '..'), { recursive: true });
  writeFileSync(resolve(OUT), JSON.stringify(report, null, 2));
 }
 process.exit(report.ok ? 0 : 1);
}
void main();
// tools/scan-wordlist.ts
/**
 * 合规词表扫描器（CI 门禁）：扫描代码内文案（TS 字符串/模板字面量）、JSON 配置值、全部文件名。
```

```
 * 命中禁用词输出 文件:行号 并 exit 1。词表来自 packages/shared（单一数据源），追加即生效。
 *
 *   npm run scan              # 扫描整个仓库
 *   npm run scan -- <path...>   # 扫描指定路径（夹具测试用）
 *   npm run scan -- --json=artifacts/verification/scan-result.json
 */
import { mkdirSync, readFileSync, readdirSync, statSync, writeFileSync } from 'node:fs';
import { dirname, join, relative, resolve, sep } from 'node:path';
import { findBanned } from '../packages/shared/src/text-filter';
export const ROOT = resolve(import.meta.dirname, '..');
/** 不扫描的目录/文件（内部工程语言、文档、生成物、词表本身、故意含禁用词的测试）。 */
export const IGNORE = [
  'node_modules', 'package-lock.json', '.git', 'artifacts', 'dist', 'docs', 'openspec', '.opencode', '.github',
  'client/library', 'client/temp', 'client/local', 'client/build', 'client/profiles',
  'packages/shared/src/wordlist-data.ts',
  'client/assets/scripts/generated/shared/wordlist-data.ts',
];
const IGNORE_DIR_NAMES = new Set(['test', 'fixtures']);
const TEXT_EXT = new Set(['.ts', '.json', '.txt', '.html', '.yml', '.yaml']);
export interface ScanHit {
  file: string;
  line: number;
  term: string;
  category: string;
  kind: 'string' | 'json' | 'filename' | 'text';
  excerpt: string;
}
export interface Literal {
  text: string;
  line: number;
}
/** 提取 TS 源码中的字符串与模板字面量（跳过注释与正则字面量之外的代码）。 */
export function extractLiterals(src: string): Literal[] {
  const out: Literal[] = [];
  let i = 0;
  let line = 1;
  const n = src.length;
  while (i < n) {
    const c = src[i]!;
    const next = src[i + 1];
    if (c === '\n') {
      line++;
      i++;
    } else if (c === '/' && next === '/') {
      while (i < n && src[i] !== '\n') i++;
    } else if (c === '/' && next === '*') {
      i += 2;
      while (i < n && !(src[i] === '*' && src[i + 1] === '/')) {
        if (src[i] === '\n') line++;
        i++;
```

```
    }
    i += 2;
   } else if (c === '"' || c === "'" || c === '`') {
    const startLine = line;
    let text = '';
    i++;
    let depth = 0;
    while (i < n) {
     const ch = src[i]!;
     if (ch === '\\') {
      text += src[i + 1] ?? '';
      i += 2;
      continue;
     }
     if (ch === '\n') line++;
     if (c === '`' && ch === '$' && src[i + 1] === '{') {
      depth++;
      text += '{';
      i += 2;
      continue;
     }
     if (c === '`' && depth > 0) {
      if (ch === '}') depth--;
      i++;
      continue;
     }
     if (ch === c) break;
     text += ch;
     i++;
    }
    i++;
    if (text.trim()) out.push({ text, line: startLine });
   } else {
    i++;
   }
  }
  return out;
}
function jsonStrings(value: unknown, acc: string[] = []): string[] {
  if (typeof value === 'string') acc.push(value);
  else if (Array.isArray(value)) value.forEach((v) => jsonStrings(v, acc));
  else if (value && typeof value === 'object') Object.values(value).forEach((v) => jsonStrings(v, acc));
  return acc;
}
function lineOf(src: string, needle: string): number {
  const idx = src.indexOf(needle);
  return idx < 0 ? 1 : src.slice(0, idx).split('\n').length;
}
function isIgnored(rel: string): boolean {
  const norm = rel.split(sep).join('/');
```

```
  return IGNORE.some((p) => norm === p || norm.startsWith(`${p}/`));
}
export function walk(dir: string, root = ROOT, acc: string[] = []): string[] {
  for (const name of readdirSync(dir)) {
    const full = join(dir, name);
    const rel = relative(root, full);
    if (isIgnored(rel)) continue;
    const st = statSync(full);
    if (st.isDirectory()) {
      if (!IGNORE_DIR_NAMES.has(name)) walk(full, root, acc);
    } else acc.push(full);
  }
  return acc;
}
export function scanFile(full: string, root = ROOT): ScanHit[] {
  const rel = relative(root, full).split(sep).join('/');
  const hits: ScanHit[] = [];
  const base = rel.split('/').pop() ?? rel;
  for (const h of findBanned(base.replace(/\.[^.]+$/, '').replace(/[-_.]/g, ' '))) {
    hits.push({ file: rel, line: 0, term: h.term, category: h.category, kind: 'filename', excerpt: base });
  }
  const ext = base.includes('.') ? base.slice(base.lastIndexOf('.')) : '';
  if (!TEXT_EXT.has(ext)) return hits;
  const src = readFileSync(full, 'utf8');
  if (ext === '.ts') {
    for (const lit of extractLiterals(src)) {
      for (const h of findBanned(lit.text)) hits.push({ file: rel, line: lit.line, term: h.term, category: h.category, kind: 'string', excerpt: lit.text.slice(0,
60) });
    }
  } else if (ext === '.json') {
    let parsed: unknown;
    try {
      parsed = JSON.parse(src);
    } catch {
      parsed = src;
    }
    for (const s of jsonStrings(parsed)) {
      for (const h of findBanned(s)) hits.push({ file: rel, line: lineOf(src, s), term: h.term, category: h.category, kind: 'json', excerpt: s.slice(0, 60) });
    }
  } else {
    src.split('\n').forEach((text, i) => {
      for (const h of findBanned(text)) hits.push({ file: rel, line: i + 1, term: h.term, category: h.category, kind: 'text', excerpt: text.slice(0, 60) });
    });
  }
  return hits;
}
export function scan(paths: string[], root = ROOT): { files: number; hits: ScanHit[] } {
  const files = paths.flatMap((p) => (statSync(p).isDirectory() ? walk(p, root) : [p]));
  return { files: files.length, hits: files.flatMap((f) => scanFile(f, root)) };
}
function main(): void {
```

```
 const args = process.argv.slice(2);
 const jsonOut = args.find((a) => a.startsWith('--json='))?.slice(7);
 const targets = args.filter((a) => !a.startsWith('--')).map((p) => resolve(p));
 const result = scan(targets.length ? targets : [ROOT]);
 if (jsonOut) {
  mkdirSync(dirname(resolve(jsonOut)), { recursive: true });
  writeFileSync(resolve(jsonOut), JSON.stringify({ ok: result.hits.length === 0, ...result }, null, 2));
 }
 if (result.hits.length === 0) {
  console.log(`wordlist scan: ${result.files} files, 0 hits`);
  return;
 }
 for (const h of result.hits) console.error(`${h.file}:${h.line}  [${h.category}] "${h.term}"  (${h.kind}) ${h.excerpt}`);
 console.error(`wordlist scan: ${result.hits.length} hit(s) in ${result.files} files`);
 process.exit(1);
}
if (process.argv[1] && resolve(process.argv[1]) === resolve(import.meta.filename)) main();
// tools/soak.ts
/**
 * 无头长稳抽测（任务 5.5 的代理指标）：客户端逻辑层（DemoSource + ViewModel）以 60fps 虚拟节拍
 * 连续运行 30 分钟（虚拟时间，加速执行），统计每帧耗时（最大/P99）与堆内存增长。
 * 不含渲染开销——真实帧率/内存仍需人在编辑器/exe 中抽验（任务 5.5）。
 *
 *   npm run soak -- --minutes=30 --json=artifacts/verification/soak.json
 */
import { mkdirSync, writeFileSync } from 'node:fs';
import { dirname, resolve } from 'node:path';
// client/ 的 package.json 为 Cocos 工程（CommonJS），这里用动态导入兼容命名导出的互操作
const interop = <T>(m: { default?: T } & T): T => (m.default && typeof m.default === 'object' ? m.default : m);
const { DemoSource } = interop(await import('../client/assets/scripts/logic/sources'));
const { ViewModel } = interop(await import('../client/assets/scripts/logic/view-model'));
const arg = (n: string, d: string) => process.argv.find((a) => a.startsWith(`--${n}=`))?.slice(n.length + 3) ?? d;
const MINUTES = Number(arg('minutes', '30'));
const OUT = arg('json', '');
const FRAME_MS = 1000 / 60;
const src = new DemoSource({ seed: 42, speed: 1 });
const vm = new ViewModel({ A: '锦鲤', B: '神龟' });
const costs: number[] = [];
const heap: { minute: number; mb: number }[] = [];
let matches = 0;
let lastMatch = 1;
const gc = (globalThis as { gc?: () => void }).gc;
const heapMb = () => {
 gc?.();
 return process.memoryUsage().heapUsed / 1024 / 1024;
};
const totalFrames = Math.floor((MINUTES * 60 * 1000) / FRAME_MS);
heap.push({ minute: 0, mb: heapMb() });
for (let i = 0; i < totalFrames; i++) {
 const t = i * FRAME_MS;
```

```
  const t0 = performance.now();
  const f = src.poll(t);
  if (f) vm.apply(f.snapshot, f.effects);
  costs.push(performance.now() - t0);
  if (src.matchNo !== lastMatch) {
    matches++;
    lastMatch = src.matchNo;
  }
  if (i > 0 && i % (60 * 60 * 5) === 0) heap.push({ minute: Math.round((t / 60000) * 10) / 10, mb: heapMb() });
}
heap.push({ minute: MINUTES, mb: heapMb() });
const sorted = [...costs].sort((a, b) => a - b);
const p99 = sorted[Math.floor(sorted.length * 0.99)]!;
const max = sorted.at(-1)!;
const first = heap[1]?.mb ?? heap[0]!.mb;
const growth = heap.at(-1)!.mb - first;
const report = {
  virtualMinutes: MINUTES,
  frames: totalFrames,
  matchesCompleted: matches,
  frameCostMs: { p99: Math.round(p99 * 1000) / 1000, max: Math.round(max * 1000) / 1000, budget: 33.3 },
  heapMb: heap.map((h) => ({ ...h, mb: Math.round(h.mb * 10) / 10 })),
  heapGrowthAfterWarmupMb: Math.round(growth * 10) / 10,
  gcExposed: Boolean(gc),
  ok: max < 33.3 && growth < 20 && matches >= 1,
};
console.log(JSON.stringify(report, null, 2));
if (OUT) {
  mkdirSync(dirname(resolve(OUT)), { recursive: true });
  writeFileSync(resolve(OUT), JSON.stringify(report, null, 2));
}
process.exit(report.ok ? 0 : 1);
// tools/softcopyright-pdf.ts
/**
 * 软著源代码 PDF 排版（任务 6.1，D1 交付件）。
 * 规范：A4 纵向；页眉=软件全称+版本；页脚=著作权人；右上角页码；每页 50 行；
 * 去空行；前 30 页 + 后 30 页（不足 60 页则全量）。默认排除 V0 专用的 local-grab 与生成物（R3b）。
 *
 *   npm run softcopyright                  # 输出 artifacts/softcopyright/source-code.pdf
 *   npm run softcopyright -- --config=path.json --out=path.pdf
 */
import { createWriteStream, existsSync, mkdirSync, readFileSync, readdirSync, statSync } from 'node:fs';
import { dirname, join, relative, resolve } from 'node:path';
import PDFDocument from 'pdfkit';
const ROOT = resolve(import.meta.dirname, '..');
export interface SoftCopyrightConfig {
  softwareName: string;
  version: string;
  owner: string;
  include: string[];
```

```
  exclude: string[];
  linesPerPage: number;
  pagesEachEnd: number;
  font: string;
  fontFamily: string;
}
export function collectFiles(cfg: SoftCopyrightConfig, root = ROOT): string[] {
  const excluded = cfg.exclude.map((e) => resolve(root, e));
  const out: string[] = [];
  const visit = (p: string) => {
    if (excluded.some((e) => p === e || p.startsWith(e + '/'))) return;
    const st = statSync(p);
    if (st.isDirectory()) readdirSync(p).sort().forEach((n) => visit(join(p, n)));
    else if (p.endsWith('.ts')) out.push(p);
  };
  for (const inc of cfg.include) if (existsSync(resolve(root, inc))) visit(resolve(root, inc));
  return out;
}
/** 有效代码行：去空行；行首加相对路径标记不计入（仅文件首行带路径注释）。 */
export function collectLines(files: string[], root = ROOT): string[] {
  const lines: string[] = [];
  for (const f of files) {
    lines.push(`// ${relative(root, f)}`);
    for (const l of readFileSync(f, 'utf8').split('\n')) {
      const t = l.replace(/\s+$/, '').replace(/\t/g, '  ');
      if (t.trim() !== '') lines.push(t);
    }
  }
  return lines;
}
export function paginate(lines: string[], perPage: number, endPages: number): { pages: string[][]; total: number; truncated: boolean } {
  const all: string[][] = [];
  for (let i = 0; i < lines.length; i += perPage) all.push(lines.slice(i, i + perPage));
  if (all.length <= endPages * 2) return { pages: all, total: all.length, truncated: false };
  // 后 30 页从文末倒着切，保证结束页是满页（登记规范：结束页 ≥15 行）
  const tail: string[][] = [];
  for (let end = lines.length; tail.length < endPages; end -= perPage) tail.unshift(lines.slice(end - perPage, end));
  return { pages: [...all.slice(0, endPages), ...tail], total: all.length, truncated: true };
}
async function render(cfg: SoftCopyrightConfig, pages: string[][], outPath: string): Promise<void> {
  mkdirSync(dirname(outPath), { recursive: true });
  const doc = new PDFDocument({ size: 'A4', margins: { top: 0, bottom: 0, left: 0, right: 0 }, autoFirstPage: false });
  const stream = createWriteStream(outPath);
  doc.pipe(stream);
  doc.registerFont('cjk', cfg.font, cfg.fontFamily);
  const header = `${cfg.softwareName} ${cfg.version}`;
  const lineH = (842 - 56 - 48) / cfg.linesPerPage;
  pages.forEach((lines, idx) => {
    doc.addPage();
    doc.font('cjk').fontSize(9);
```

```
    doc.text(header, 48, 28, { width: 400, lineBreak: false });
    doc.text(String(idx + 1), 595 - 48 - 60, 28, { width: 60, align: 'right', lineBreak: false });
    doc.moveTo(48, 44).lineTo(595 - 48, 44).lineWidth(0.5).stroke();
    doc.fontSize(7.5);
    lines.forEach((l, i) => doc.text(l, 48, 56 + i * lineH, { width: 595 - 96, lineBreak: false }));
    doc.fontSize(8).text(cfg.owner, 48, 842 - 34, { width: 595 - 96, align: 'center', lineBreak: false });
  });
  doc.end();
  await new Promise<void>((r) => stream.on('finish', () => r()));
}
async function main(): Promise<void> {
  const arg = (n: string) => process.argv.find((a) => a.startsWith(`--${n}=`))?.slice(n.length + 3);
  const cfgPath = resolve(arg('config') ?? join(ROOT, 'tools/softcopyright.config.json'));
  const cfg = JSON.parse(readFileSync(cfgPath, 'utf8')) as SoftCopyrightConfig;
  const out = resolve(arg('out') ?? join(ROOT, 'artifacts/softcopyright/source-code.pdf'));
  const files = collectFiles(cfg);
  const lines = collectLines(files);
  const { pages, total, truncated } = paginate(lines, cfg.linesPerPage, cfg.pagesEachEnd);
  await render(cfg, pages, out);
  console.log(`softcopyright: ${files.length} files, ${lines.length} lines, ${total} pages total -> ${pages.length} pages${truncated ? ' (first/last 30)' : ''}\n  ${out}`);
  if (lines.length < 3000) console.warn(`WARNING: ${lines.length} lines < 3000 (软著要求源程序 ≥3000 行)`);
}
if (process.argv[1] && resolve(process.argv[1]) === resolve(import.meta.filename)) void main();
// tools/sync-client.ts
/**
 * shared/core → Cocos 客户端单向同步：拷贝源码到 client/assets/scripts/generated/，
 * 把包导入改写为相对路径。客户端内不得手改生成物；--check 校验一致（CI 门禁）。
 */
import { existsSync, mkdirSync, readFileSync, readdirSync, rmSync, writeFileSync } from 'node:fs';
import { dirname, join, resolve } from 'node:path';
const ROOT = resolve(import.meta.dirname, '..');
const OUT = join(ROOT, 'client/assets/scripts/generated');
const HEADER = '// AUTO-GENERATED by tools/sync-client.ts — do not edit. Source: ';
const SOURCES = [
  { from: 'packages/shared/src', to: 'shared', skip: new Set(['schema.ts']) },
  { from: 'packages/core/src', to: 'core', skip: new Set<string>() },
];
export function buildOutputs(): Map<string, string> {
  const files = new Map<string, string>();
  for (const s of SOURCES) {
    for (const name of readdirSync(join(ROOT, s.from)).sort()) {
      if (!name.endsWith('.ts') || s.skip.has(name)) continue;
      const src = readFileSync(join(ROOT, s.from, name), 'utf8').replace(/from '@jinli\/shared'/g, "from '../shared'");
      files.set(join(s.to, name), `${HEADER}${s.from}/${name}\n${src}`);
    }
  }
  return files;
}
function listExisting(dir: string, base = dir, acc: string[] = []): string[] {
  if (!existsSync(dir)) return acc;
```

```
  for (const e of readdirSync(dir, { withFileTypes: true })) {
    const full = join(dir, e.name);
    if (e.isDirectory()) listExisting(full, base, acc);
    else if (e.name.endsWith('.ts')) acc.push(full.slice(base.length + 1));
  }
  return acc;
}
function main(): void {
  const check = process.argv.includes('--check');
  const jsonOut = process.argv.find((a) => a.startsWith('--json='))?.slice(7);
  const outputs = buildOutputs();
  const problems: string[] = [];
  for (const [rel, content] of outputs) {
    const target = join(OUT, rel);
    const current = existsSync(target) ? readFileSync(target, 'utf8') : null;
    if (current === content) continue;
    if (check) problems.push(`${current === null ? 'missing' : 'stale'}: ${rel}`);
    else {
      mkdirSync(dirname(target), { recursive: true });
      writeFileSync(target, content);
    }
  }
  for (const rel of listExisting(OUT)) {
    if (outputs.has(rel)) continue;
    if (check) problems.push(`orphan: ${rel}`);
    else rmSync(join(OUT, rel));
  }
  if (jsonOut) {
    mkdirSync(dirname(resolve(jsonOut)), { recursive: true });
    writeFileSync(resolve(jsonOut), JSON.stringify({ ok: problems.length === 0, files: outputs.size, problems }, null, 2));
  }
  if (check && problems.length) {
    console.error(`client sync check failed (run npm run sync-client):\n  ${problems.join('\n  ')}`);
    process.exit(1);
  }
  console.log(check ? `client sync check: ${outputs.size} files in sync` : `client sync: ${outputs.size} files written`);
}
main();
// tools/verify-pack.ts
/**
 * 验证包组装（任务 7.2）：汇总 artifacts/verification/ 下机器可读结果 → VERIFICATION.md。
 * 先 `npm run ci`，再（可选）`npm run loadtest -- --json=artifacts/verification/loadtest.json`。
 */
import { existsSync, readFileSync, writeFileSync } from 'node:fs';
import { resolve } from 'node:path';
const OUT = resolve(import.meta.dirname, '../artifacts/verification');
const read = <T>(name: string): T | null => {
  const p = resolve(OUT, name);
  return existsSync(p) ? (JSON.parse(readFileSync(p, 'utf8')) as T) : null;
};
```

```
const manifest = read<{ ok: boolean; finishedAt: string; node: string; gates: { name: string; ok: boolean; ms: number }[] }>('manifest.json');
const tests = read<{ numTotalTests: number; numPassedTests: number; numFailedTests: number; numTotalTestSuites: number }>('test-report.json');
const scan = read<{ ok: boolean; files: number; hits: unknown[] }>('scan-result.json');
const sync = read<{ ok: boolean; files: number }>('sync-check.json');
const soak = read<{ virtualMinutes: number; frames: number; matchesCompleted: number; frameCostMs: { p99: number; max: number; budget: number }; heapGrowthAfterWarmupMb: number; ok: boolean }>('soak.json');
const load = read<{ qps: number; seconds: number; sent: number; delivered: number; lost: number; p995Ms: number; ok: boolean }>('loadtest.json');
const mark = (ok: boolean | undefined) => (ok === undefined ? '☒ 未运行' : ok ? '☑' : '☒');
const md = `# 验证包 · tech-foundation

> 生成时间 ${new Date().toISOString()} · CI 完成于 ${manifest?.finishedAt ?? '（未运行 npm run ci）'} · Node ${manifest?.node ?? '-'}

> 先看本页，再按"抽验点"人工验证。

## 1. 自测报告

| 门禁 | 结果 | 说明 |

| --- | --- | --- |

${(manifest?.gates ?? []).map((g) => `| ${g.name} | ${mark(g.ok)} | ${(g.ms / 1000).toFixed(1)}s |`).join('\n') || '| - | ☒ | 先运行 npm run ci |'}

- 单元/集成测试：${tests ? `${tests.numPassedTests}/${tests.numTotalTests} 通过（${tests.numTotalTestSuites} 个套件，失败 ${tests.numFailedTests}）` : '未运行'}
- 管道压测（任务 3.6，进程内）：${load ? `${mark(load.ok)} ${load.qps} QPS × ${load.seconds}s，发送 ${load.sent}，投递 ${load.delivered}，丢失 ${load.lost}，P99 ${load.p995Ms}ms（红线 <1000ms）` : '☒ 未运行（npm run loadtest -- --json=artifacts/verification/loadtest.json）'}
- 客户端逻辑层长稳（任务 5.5 代理指标，无渲染）：${soak ? `${mark(soak.ok)} 虚拟 ${soak.virtualMinutes} 分钟 @60fps（${soak.frames} 帧，完成 ${soak.matchesCompleted} 局），帧耗时 P99 ${soak.frameCostMs.p99}ms / 最大 ${soak.frameCostMs.max}ms（预算 ${soak.frameCostMs.budget}ms），堆内存增长 ${soak.heapGrowthAfterWarmupMb}MB` : '☒ 未运行（npm run soak -- --json=artifacts/verification/soak.json）'}
- 客户端同步一致性：${mark(sync?.ok)} ${sync ? `${sync.files} 个文件` : ''}

## 2. 合规自查（对照 config.yaml rules 与方案附录 C）

| 条目 | 结论 | 证据 |

| --- | --- | --- |

| R1 随机产出不可交易（代码级） | ${mark(manifest?.gates.find((g) => g.name === 'test')?.ok)} | core 纯度测试：无 trade/transfer/redeem 接口；T4 随机仅外观（效果固定），概率合计 100 且已入配置 |
| R1 无现金/实物回报 | ☒ | core 输出类型无任何金额/奖品字段（"never outputs currency fields" 测试） |
| R2 零金额显示 | ${mark(manifest?.gates.find((g) => g.name === 'test')?.ok)} | 金额止于管道（normalizer 审计回调）；客户端 feed 零货币单位测试 |
| R2 催氪红线进词表 | ${mark(scan?.ok)} | 词表含催付指令、金额播报、对比羞辱、收益承诺等催氪句式；扫描 ${scan ? `${scan.files} 文件，${scan.hits.length} 命中` : '未运行'} |
| R3 平台无关 | ${mark(manifest?.gates.find((g) => g.name === 'lint')?.ok)} | core/shared 禁止 import 平台/IO 模块（eslint + 依赖扫描测试） |
| R3b 民间路线边界 | ☒ | local-grab 独立目录、软著 PDF 默认排除、main.ts 启动时打印 V0 声明 |
| R4 预算 | ☒ | 本变更零现金支出 |
| 内部代号不外显 | ${mark(scan?.ok)} | 内部代号已进词表（含大小写变体）；软件名为"幸运锦鲤直播互动系统" |
| 单件礼物进度 ≤12.5% | ☒ | zod schema 校验 + 单测 |

## 3. 风险声明

1. **抓取消息格式已按工具官方文档对齐**（扁平 JSON、ws://127.0.0.1:1088/ws/<直播间标识>、ping/pong、system 状态帧、snake/camel 键容错），但**未经真机核实**：工具文档自述礼物消息可能缺失/字段不全——缺礼物计价信息时按 T0 兜底，真机联调（4.3）后可能仍需微调（4.4）。
2. **Cocos 视图层未经编辑器编译**——CI 仅用 cc 类型桩做类型检查（typecheck-view），不等于真实编译；逻辑层已单测，视图层需人在编辑器打开验证（5.1）。
3. **数值 V1 + v0.7 进度上限未经真实数据校准**——T5+ 进度/元显著低于 T0~T4，对 H3 偏保守；属 L2 数值方向审批。
4. **压测为进程内**，不含网络与全局 4200 QPS——后置 compliance-testkit。
5. **软著源码行数**：见 \`npm run softcopyright\` 输出；著作权人姓名需人填写（tools/softcopyright.config.json）。
6. **Goal v1.2 修订未经人签署**：config.yaml 的修订属 Goal 修订，需人 L3 确认。

## 4. 验证级别建议

| 任务 | 级别 | 人抽验点 |

| --- | --- | --- |

| 4.3 真机联调 | L2 | 见 tasks.md 4.3 清单 |
| 5.1 Cocos 工程 | L2 | 见 tasks.md 5.1 清单 |
| 5.5 性能抽测 | L2 | 见 tasks.md 5.5 清单 |
| 6.2 软著名称与 PDF | **L3** | 见 tasks.md 6.2 清单（提交前全验） |
| 7.3 架构评审 | **L3** | 对照 design AD1~AD8 |
| 数值/进度上限方向 | L2 | docs 方案 5.9 ⑧ |
| Goal v1.2 修订 | **L3** | docs/GOAL-阶段计划.md 末尾修订记录 |

`;
writeFileSync(resolve(OUT, '../VERIFICATION.md'), md);
console.log('verification pack written: artifacts/VERIFICATION.md');
```

（著作权人姓名，提交前由人填写）